TECHNICAL SPECIFICATION

Midgard L2: Scaling Cardano with Optimistic Rollups

WORKING DRAFT

2025-01-18

FluidTokens ANASTASIA LABS

Midgard L2: Scaling Cardano with Optimistic Rollups

Philip DiSarro
Anastasia Labs

Jonathan Rodriguez
Anastasia Labs
info@anastasialabs.com

George Flerovsky

Abstract

Midgard is the first optimistic rollup protocol on Cardano. It offloads transaction processing from Layer 1 (L1) to Layer 2 (L2) while maintaining L1's security and decentralization properties for those transactions. As a result, it can handle a significantly higher volume of transactions without compromising on security.

This whitepaper outlines the interactions between L1 and L2, the role of operators and watchers, and the processes for managing state transitions and resolving disputes. The optimistic rollup model implemented in Midgard represents a significant advancement in the scalability and security of general-purpose Layer 2 solutions on Cardano.

We discuss the economic incentives that ensure the protocol's integrity and efficiency. Operators guarantee the validity of their committed blocks by posting bonds, which are slashable if the blocks are proven invalid. Members of the public are incentivized to watch the operators' blocks, and they are rewarded with a portion of the slashed bonds when their fraud proofs prevent invalid blocks from merging into Midgard's confirmed state.

Midgard's architecture leverages a family of smart contracts on Cardano's L1 to manage state transitions and enforce the security of the L2 operations. Key components include a robust operator management system, efficient storage for transaction blocks committed from the L2, and a comprehensive mechanism for submitting and validating fraud proofs. Overall, Midgard's design aims to provide a scalable and secure solution for the growing needs of the Cardano ecosystem.

Contributors

Raul Antonio
Fluid Tokens

Matteo Coppola
Fluid Tokens

fallen-icarus
P2P-Defi

Riley Kilgore
IOG

Keyan Maskoot
Anastasia Labs

Bora Oben
Anastasia Labs

Mark Petruska
Anastasia Labs

Kasey White
Cardano Foundation

This work is licensed under a Creative Commons "Attribution-ShareAlike 4.0 International" license.

Contents

Introduction 1

1 Ledger state 3
- 1.1 Block . . . 4
- 1.2 Utxo set . . . 6
- 1.3 Deposit event . . . 6
- 1.4 Withdrawal event . . . 7
- 1.5 Transaction . . . 8
- 1.6 Confirmed state . . . 12

2 User event protocol 14
- 2.1 Deposit (L1) . . . 14
- 2.2 Transaction request (L2) . . . 16
- 2.3 Transaction order (L1) . . . 17
- 2.4 Withdrawal order (L1) . . . 19

3 Consensus protocol 21
- 3.1 Time model . . . 22
- 3.2 Operator directory . . . 23
- 3.3 Scheduler . . . 31
- 3.4 State queue . . . 34
- 3.5 Escape hatch . . . 37
- 3.6 Settlement queue . . . 38
- 3.7 Reserve . . . 39
- 3.8 Midgard hub oracle . . . 40

4 Proof protocol 42
- 4.1 Fraud proof catalogue . . . 42
- 4.2 Fraud proof tokens . . . 44
- 4.3 Fraud proof computation threads . . . 45

5 Ledger rules and fraud proofs 48
- 5.1 Midgard Ledger Rules and Fraud Proofs . . . 48
- 5.2 Custom Midgard ledger rules . . . 56
- 5.3 Data availability rules . . . 57

6 Offchain data architecture 58
- 6.1 Data availability layer . . . 58
- 6.2 Archive node . . . 59

7 Phase Two Validation **61**

7.1 Overview 61

7.2 State Representation 62

7.3 Off-chain Decoding 63

7.4 Fraud Proofs in UPLC Evaluation 64

A General onchain data structures and mechanisms **66**

A.1 Linked list 66

A.2 Single-threaded state machine 79

B Midgard L1 transactions **83**

C Design and implementation considerations **84**

C.1 Protocol parameters 84

C.2 Protocol security 86

List of figures **88**

Introduction

Midgard is a Layer 2 (L2) scaling solution for the Cardano blockchain. It employs optimistic rollup technology to enhance Cardano's capacity to process transactions and host more complex applications, delivering a richer user experience at a more competitive cost. As Cardano continues to grow in usage and demand, scaling solutions like Midgard are critical for maintaining high performance and low transaction costs. This whitepaper describes the architecture and technical design of the Midgard protocol, detailing how it integrates with Cardano's Layer 1 (L1) to deliver secure and efficient transaction processing.

Optimistic rollups process blocks of transactions off-chain and commit those blocks' headers onchain to the L1 ledger. Each block is committed by a Midgard operator who guarantees the block's validity. The block then waits in a queue for at least a fixed duration to be merged into the confirmed state of the optimistic rollup on L1. The operator must collateralize their guarantee with a bond deposit and publish the full contents of the block on the publicly accessible data availability (DA) layer.

While a committed block is queued, anyone can inspect its contents on the data availability layer and ascertain whether it is valid. If someone detects that the block is invalid, they can submit a fraud proof to prevent it from being merged into the confirmed state, slash the operator's bond, and receive a portion of the forfeited bond as a reward. Thus, an optimistic rollup can process a large number of transactions offchain while maintaining security and finality properties that are similar to transactions processed directly onchain, as long as:

- The bond requirement for the rollup's operators is large enough to discourage fraud.
- The reward for preventing an invalid block from merging is large enough to encourage public vigilance in watching the operators.
- The waiting period for committed blocks is long enough to allow the watchers to detect and prove fraud before those blocks are merged.
- The data availability layer is accessible by anyone who wishes to inspect the rollup blocks at any time that they wish to do so.

Whenever the latter three security parameter values are calibrated to provide a high probability of invalid blocks being detected and disqualified, the bond requirement is a strong deterrent against operators attempting fraud. An operator cannot dismiss the forfeited bond as merely a "cost of doing business" paid to obtain potentially larger revenues from fraud. Whenever an invalid block is disqualified, it does *not* affect the confirmed state, so there are no revenues from that fraud to offset the operator's forfeited bond.

The main design goal of Midgard is to streamline the processes by which blocks are committed/merged, fraud is detected, and fraud proofs are verified onchain. Advancing this goal allows the security parameters to be calibrated to achieve a better balance between security, transaction throughput, confirmation time, and community participation in committing blocks and detecting fraud.

Scalability and efficiency

By processing transactions off-chain and only validating them on-chain when fraud proofs challenge them, Midgard significantly increases throughput and reduces costs for Cardano transactions. Its rollup blocks use sparse Merkle trees and compact state representations to enhance the protocol's efficiency further, enabling it to handle a large volume of transactions without overburdening the L1.

The deterministic nature of Cardano transactions allows Midgard fraud proofs to pinpoint the specific site of a transaction that violated Midgard's ledger rules, without having to look at any other parts of that transaction, any other unrelated transactions within the block, or any other blocks. This keeps fraud proofs and their onchain validation procedures small and efficient, which reduces the time and cost needed to submit fraud proofs when invalid blocks are detected, which makes it feasible for a wider group of people to police Midgard's blocks. In this way, Midgard significantly reduces fraud proof size relative to optimistic rollups used in Ethereum and other account-based blockchain ecosystems, where a much larger part of the global blockchain state needs to be inspected when constructing and verifying a fraud proof.

Censorship resistance and fallback mechanisms

On its own, the optimistic rollup mechanism described above ensures a high-level of assurance for the validity of block headers committed to the state queue and merged to Midgard's confirmed state. However, it does not prevent operators from censoring users' deposits, withdrawals, and L2 transactions. Consequently, Midgard's consensus protocol includes additional smart contract mechanisms to provide censorship resistance for these events.

Midgard deposits and withdrawals are initiated via L1 smart contracts that assign definite inclusion times to them. An operator block is invalid if it contains these inclusion times in its event interval but fails to include the associated deposit or withdrawal events. This ensures that if operators continue committing blocks to Midgard's state queue, then they cannot ignore deposit and withdrawal events.

Midgard L2 transaction requests are typically submitted to operators via a publicly accessible API, and they can be ignored by operators. However, any user can escalate his L2 transaction request by posting a transaction order on L1. Similar to Midgard deposits and withdrawals, an L1 transaction order is assigned an inclusion time that guarantees its inclusion in a subsequent valid block.

If Midgard operators stop committing blocks at all to the state queue, then the inclusion times on their own cannot guarantee that deposits, withdrawals, and L2 transactions will be processed in a timely manner. However, for this extreme case, Midgard's consensus protocol includes the escape hatch mechanism, which allows a special non-optimistic block to be appended to the state queue by a non-operator. This block can include any deposits, withdrawals, and L2 transactions that are verified on L1 to comply with Midgard's ledger rules. This ensures that user funds cannot be stranded on Midgard even if its operators entirely stop committing blocks.

Feedback and contributions welcome!

This document specifies the ledger, onchain contracts, and offchain components of the Midgard architecture. We detail the roles of users, operators, and watchers, how blocks are committed and merged on L1, and how fraud proofs are handled.

We invite developers, validators, and the broader community to engage with this document and offer feedback and contributions to help refine and enhance the protocol.

Chapter 1

Ledger state

Midgard's L2 ledger consists of a chain of blocks. Each block defines a transition from the previous block's set of unspent transaction outputs (utxos) to a new utxo set. Unlike Cardano's closed-system L1 ledger, Midgard's open-system L2 ledger allows block transitions to create and destroy utxos in response to exogenous events—namely, deposits and withdrawal requests that occur on the L1 ledger. This is in addition to Midgard's endogenous L2 transactions, which work the same as L1 transactions but with reduced functionality (no staking/governance actions).[1]

The blocks are stored temporarily on Midgard's data availability layer and permanently on Midgard's archive nodes. The blocks' headers are committed to Midgard's L1 state queue data structure to establish immutability for the blocks' sequence and contents as part of Midgard's L1 contract-based consensus protocol. Block headers have a fixed byte size regardless of how many deposit, transaction, and withdrawal events are held by their blocks—this size leverage between blocks and headers is how Midgard multiplies Cardano's transaction throughput.

Midgard L1 contract-based consensus protocol irreversibly considers a block to be confirmed as soon as all its predecessors are confirmed and one of the following holds:

Optimistic confirmation. The block's maturity period has elapsed without any fraud proof being verified on L1 to prove that the block violates one of Midgard's ledger rules.

Non-optimistic confirmation. A compliance proof has been verified on L1 to prove that the block complies with all Midgard ledger rules.

The irreversibility of block confirmation allows the L1 representation of confirmed block headers to be condensed. For instance, it can stop tracking confirmed withdrawal requests after they are paid out and confirmed deposits after they are absorbed into Midgard's reserve. It can avoid tracking any confirmed transactions and only needs to track the last confirmed block's utxo set, as it is required to confirm the next block. Finally, it can drop all other header data for the last confirmed block's predecessors, as it is implicitly tracked by a chained hash in the last confirmed header and is not required to confirm the next block.

As a result, Midgard's L1 confirmed state consists of a fixed-byte-size record with selected fields from the last confirmed block header and a variable-size dataset tracking confirmed withdrawal requests and deposits until they are processed.

[1] Cardano developers concerned about the absence of the "Withdraw 0" action need not worry. Midgard's ledger rules permit the "Observe" script purpose defined in CIP-112, which is a more principled replacement for "Withdraw 0" that is expected to arrive in the next era of Cardano mainnet in 2025.

1.1 Block

A block consists of a header hash, a header, and a block body:

$$\text{Block} := \left\{ \begin{array}{ll} \text{header\_hash}: & \text{HeaderHash} \\ \text{header}: & \text{Header} \\ \text{block\_body}: & \text{BlockBody} \end{array} \right\}$$

The block body contains the block's transactions, deposits, and withdrawals, along with the unspent outputs that result from applying the block's transition to the previous block's unspent outputs.

$$\text{BlockBody} := \left\{ \begin{array}{ll} \text{utxos}: & \text{UtxoSet} \\ \text{transactions}: & \text{TxSet} \\ \text{deposits}: & \text{DepositSet} \\ \text{withdrawals}: & \text{WithdrawalSet} \end{array} \right\}$$

Figure 1.1 shows how the block's `utxos` are derived from the previous block's `utxos` by applying the block's events. Withdrawals are applied before transactions, which are applied before deposits.

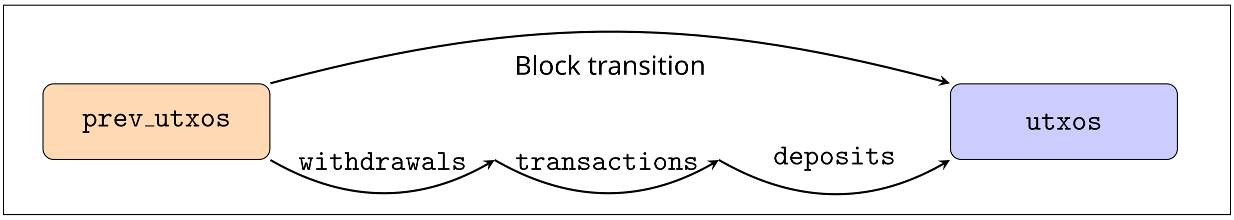

Figure 1.1: A block's transition from a previous block's utxo set to a new utxo set.

The block is what gets serialized and stored on Midgard's data availability layer. During serialization, each of the block body's sets is serialized as a sequence of pairs, sorted in ascending order on the unique key of each element.

However, only the header hash and header are stored on Cardano L1. This is sufficient because the header specifies Merkle Patricia Trie (MPT) root hashes for each of the sets in the block body. Each of these root hashes can be verified onchain by streaming over the corresponding set's elements, hashing them, and iteratively calculating the root hash.

Figure 1.2 shows an example MPT representation of a block's transactions. The $(\text{TxId}_i, \text{MidgardTx}_i)$ pairs are the data blocks of the tree, which are individually hashed to form the leaves L_i of the tree. The hashes of sibling leaves are concatenated and hashed to form their parent nodes. The nodes N_{ij} are formed by concatenating and hashing their children's hashes. The `transactions_root` is formed by concatenating and hashing its children's hashes.

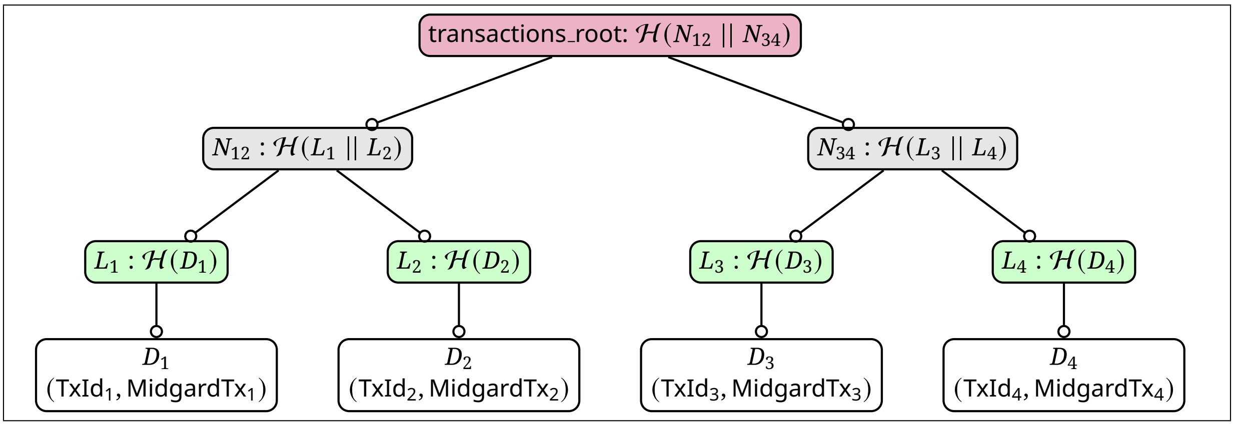

Figure 1.2: A Merkle Patricia Trie example for a block's transactions.

1.1.1 Block header

A block header is a record with fixed-size fields: integers, hashes, and fixed-size bytestrings. A block header hash is 28 bytes in size and calculated via the Blake2b-224.

$$\mathsf{HeaderHash} := \mathcal{H}_{\mathsf{Blake2b-224}}(\mathsf{Header})$$

$$\mathsf{Header} := \left\{ \begin{array}{ll} \mathsf{prev\_utxos\_root} : & \mathsf{RootHash} \\ \mathsf{utxos\_root} : & \mathsf{RootHash} \\ \mathsf{transactions\_root} : & \mathsf{RootHash} \\ \mathsf{deposits\_root} : & \mathsf{RootHash} \\ \mathsf{withdrawals\_root} : & \mathsf{RootHash} \\ \mathsf{start\_time} : & \mathsf{PosixTime} \\ \mathsf{end\_time} : & \mathsf{PosixTime} \\ \mathsf{prev\_header\_hash} : & \mathsf{HeaderHash} \\ \mathsf{operator\_vkey} : & \mathsf{VerificationKey} \\ \mathsf{protocol\_version} : & \mathsf{Int} \end{array} \right\}$$

These header fields are interpreted as follows:

- The `*_root` fields are the MPT root hashes of the corresponding sets in the block body.
- The `prev_utxos_root` is a copy of the `utxos_root` from the previous block, included for convenience in the fraud proof verification procedures.
- The `start_time` and `end_time` fields are the block's event interval bounds (see Section 3.1).
- The `prev_header_hash` field is a hash of the previous block header. For the genesis block, this field is set to 28 `0x00` bytes.
- The `operator_vkey` field is the cryptographic verification key for signatures of the operator who committed the block header to the L1 state queue.
- The `protocol_version` is the Midgard protocol version that applies to this block.

1.2 Utxo set

A utxo set is a finite map from output reference to transaction output:

$$\begin{aligned} \text{UtxoSet} &:= \text{Map}(\text{OutputRef}, \text{Output}) \\ &:= \left\{ (k_i : \text{OutputRef}, v_i : \text{Output}) \mid \forall i \neq j.\ k_i \neq k_j \right\} \end{aligned}$$

An output reference is a tuple that uniquely identifies an output by a hash of the ledger event that created it (either a transaction or a deposit) and its index among the outputs of that event.

$$\text{OutputRef} := \left\{ \begin{array}{ll} \text{id} : & \text{TxId} \\ \text{index} : & \text{Int} \end{array} \right\}$$

An output is a tuple describing a bundle of tokens, data, and a script that have been placed by a transaction at an address in the ledger:

$$\text{Output} := \left\{ \begin{array}{ll} \text{addr} : & \text{Address} \\ \text{value} : & \text{Value} \\ \text{datum} : & \text{Option}(\text{Data}) \\ \text{script} : & \text{Option}(\text{Script}) \end{array} \right\}$$

Within the context of a Midgard block, the utxo set that we are interested in consists of the outputs created by deposits and transactions but not yet spent by transactions and withdrawals, considering all the deposits, transactions, and withdrawals of this block and all its predecessors. This is the utxo set that we transform into an MPT and place into the block body's `utxo` field.

> Midgard requires all L2 datums to be inline.

TODO

> Midgard uses a different network ID for L2 utxos' addresses.

TODO

1.3 Deposit event

A deposit set is a finite map from deposit IDs to deposit info:

$$\begin{aligned} \text{DepositSet} &:= \text{Map}(\text{DepositId}, \text{DepositInfo}) \\ &:= \left\{ (k_i : \text{DepositId}, v_i : \text{DepositInfo}) \mid \forall i \neq j.\ k_i \neq k_j \right\} \end{aligned}$$

A deposit event in a Midgard block acknowledges that a user has created an L1 utxo at the Midgard

L1 deposit address, intending to transfer that utxo's tokens to the L2 ledger.

$$\begin{aligned} \text{DepositEvent} &:= (\text{DepositId}, \text{DepositInfo}) \\ \text{DepositId} &:= \text{OutputRef} \\ \text{DepositInfo} &:= \left\{ \begin{array}{ll} \text{l2\_address}: & \text{Address} \\ \text{l2\_datum}: & \text{Option}(\text{Data}) \end{array} \right\} \end{aligned}$$

The deposit ID corresponds to one of the inputs spent by the user in the L1 transaction that created the L1 deposit utxo. This identifier is needed to find the L1 deposit utxo, ensure that deposit events are unique, and detect when an operator has fabricated a deposit event without the corresponding deposit utxo existing in the L1 ledger.

Suppose a deposit event is permitted by Midgard's ledger rules to be included in a block. In that case, its effect is to add a new L2 utxo to the block's utxo set containing the value from the L1 deposit utxo at the address (`l2_address`) and with the inline datum (`l2_datum`) specified by the user. The L2 output reference of this new utxo is as follows:

$$\text{l2\_outref}(\text{deposit\_id}) := \left\{ \begin{array}{ll} \text{id} & := \text{hash}(\text{deposit\_id}) \\ \text{index} & := 0 \end{array} \right\}$$

In other words, the L2 ledger treats the new utxo as if it was created by a notional transaction with `TxId` equal to the hash of the deposit ID.

If the block containing the deposit event is confirmed, the corresponding L1 deposit utxo may be absorbed into the Midgard reserves or used to pay for withdrawals. Section 2.1 describes the lifecycle of a deposit in further detail, including how the deposit event information is validated.

1.4 Withdrawal event

A withdrawal set is a finite map from withdrawal ID to withdrawal info:

$$\begin{aligned} \text{WithdrawalSet} &:= \text{Map}(\text{WithdrawalId}, \text{WithdrawalInfo}) \\ &:= \left\{ (k_i : \text{WithdrawalId}, v_i : \text{WithdrawalInfo}) \mid \forall i \neq j.\ k_i \neq k_j \right\} \end{aligned}$$

A withdrawal event in a Midgard block acknowledges that a user has created an L1 utxo at the Midgard L1 withdrawal address, requesting the transfer of an L2 utxo's tokens to the L1 ledger.

$$\begin{aligned} \text{WithdrawalEvent} &:= (\text{WithdrawalId}, \text{WithdrawalInfo}) \\ \text{WithdrawalId} &:= \text{OutputRef} \\ \text{WithdrawalInfo} &:= \left\{ \begin{array}{ll} \text{l2\_outref}: & \text{OutputRef} \\ \text{l1\_address}: & \text{Address} \\ \text{l1\_datum}: & \text{Option}(\text{Data}) \end{array} \right\} \end{aligned}$$

The `WithdrawalId` of a withdrawal event corresponds to one of the inputs spent by the user in the L1 transaction that created the L1 withdrawal request utxo. This key is needed to identify the L1 withdrawal utxo, ensure that withdrawal events are unique, and detect when an operator has fabricated a withdrawal event without the corresponding withdrawal request existing in the L1 ledger.

If a withdrawal event is permitted by Midgard's ledger rules to be included in a block, its effect is to remove the output at output-reference `l2_outref` from the block's utxo set.

Suppose the block containing the withdrawal event is confirmed. In that case, tokens from the Midgard reserve and confirmed deposits can be used to pay for the creation of an L1 utxo at the address (`l1_address`) and with the inline datum (`l1_datum`) specified by the user, containing the value from the withdrawn L2 utxo. The L1 withdrawal request utxo must be spent in the transaction that pays out the withdrawal.

Section 2.4 describes the lifecycle of a withdrawal request in further detail, including how the withdrawal event information is validated.

1.5 Transaction

A transaction set in Midgard is a finite map from transaction ID to Midgard L2 transaction, where the ID is also constrained to be the Blake2b-256 hash of the transaction:

$$\begin{aligned} \mathsf{TxSet} &:= \mathsf{Map}(\mathsf{TxId}, \mathsf{MidgardTx}) \\ &:= \left\{ (k_i : \mathsf{TxId}, v_i : \mathsf{MidgardTx}) \mid k_i \equiv \mathcal{H}_{\mathsf{Blake2b-256}}(v_i),\ \forall i \neq j.\ k_i \neq k_j \right\} \end{aligned}$$

An L2 transaction in a Midgard block is an endogenous event. Its corresponding utxo set transition is validated purely based on the information contained in the utxo set. This contrasts with deposit and withdrawal events, which create and spend utxos (respectively) based on information observed outside the L2 ledger.

A transaction can only spend a utxo if it satisfies the conditions of the spending validator corresponding to the utxo's payment address, and it can only mint and burn tokens by satisfying the conditions of the corresponding minting policies. Of course, users can inject information into the utxo set via redeemer arguments and output datums set in transactions, but those are still subject to the transaction scripts' conditions.

1.5.1 Cardano transaction types

Cardano's L1 transaction type (Chang 2 hardfork) served as an initial model for Midgard's L2 transaction type. In the following, field types prefixed by a question mark `?` are set to appropriate "empty" defaults during deserialization if the serialized transaction omits them:[2] [3]

$$\mathsf{CardanoTx} := \left\{ \begin{array}{ll} \mathsf{body}: & \mathsf{CardanoTxBody} \\ \mathsf{wits}: & \mathsf{CardanoTxWits} \\ \mathsf{is\_valid}: & \mathsf{Bool} \\ \mathsf{auxiliary\_data}: & ?\ \mathsf{TxMetadata} \end{array} \right\}$$

[2] In this section, we are mainly concerned with comparing Cardano and Midgard's deserialized data types. Serialization formats and conversions are addressed in Midgard's CDDL specifications.

[3] For simplicity of exposition, we omit the "era" type parameters, effectively coercing them all to the Conway era. We also simplify the script types.

$$
\mathrm{CardanoTxBody} := \left\{ \begin{array}{ll}
\mathrm{spend\_inputs}: & \mathrm{Set}(\mathrm{OutputRef}) \\
\mathrm{collateral\_inputs}: & ?\ \mathrm{Set}(\mathrm{OutputRef}) \\
\mathrm{reference\_inputs}: & ?\ \mathrm{Set}(\mathrm{OutputRef}) \\
\mathrm{outputs}: & [\mathrm{Output}] \\
\mathrm{collateral\_return}: & ?\ \mathrm{Output} \\
\mathrm{total\_collateral}: & ?\ \mathrm{Coin} \\
\mathrm{certificates}: & ?\ [\mathrm{Set}(\mathrm{Certificate})] \\
\mathrm{withdrawals}: & ?\ \mathrm{Map}(\mathrm{RewardAccount}, \mathrm{Coin}) \\
\mathrm{fee}: & \mathrm{Coin} \\
\mathrm{validity\_interval}: & \mathrm{CardanoValidityInterval} \\
\mathrm{required\_signer\_hashes}: & ?\ \mathrm{Set}(\mathrm{VKeyHash}) \\
\mathrm{mint}: & ?\ \mathrm{Value} \\
\mathrm{script\_integrity\_hash}: & ?\ \mathrm{Hash} \\
\mathrm{auxiliary\_data\_hash}: & ?\ \mathrm{Hash} \\
\mathrm{network\_id}: & ?\ \mathrm{Network} \\
\mathrm{voting\_procedures}: & ?\ \mathrm{VotingProcedures} \\
\mathrm{proposal\_procedures}: & ?\ \mathrm{Set}(\mathrm{ProposalProcedure}) \\
\mathrm{current\_treasury\_value}: & ?\ \mathrm{Coin} \\
\mathrm{treasury\_donation}: & ?\ \mathrm{Coin}
\end{array} \right\}
$$

$$
\mathrm{CardanoTxWits} := \left\{ \begin{array}{ll}
\mathrm{addr\_tx\_wits}: & ?\ \mathrm{Set}(\mathrm{VKey}, \mathrm{Signature}, \mathrm{VKeyHash}) \\
\mathrm{boot\_addr\_tx\_wits}: & ?\ \mathrm{Set}(\mathrm{BootstrapWitness}) \\
\mathrm{script\_tx\_wits}: & ?\ \mathrm{Map}(\mathrm{ScriptHash}, \mathrm{CardanoScript}) \\
\mathrm{data\_tx\_wits}: & ?\ \mathrm{TxDats} \\
\mathrm{redeemer\_tx\_wits}: & ?\ \mathrm{Redeemers}
\end{array} \right\}
$$

$$
\begin{aligned}
\mathrm{CardanoValidityInterval} &:= (\mathrm{Option}(\mathrm{Slot}), \mathrm{Option}(\mathrm{Slot})) \\
\mathrm{CardanoScript} &:= \mathrm{TimelockScript}(\mathrm{Timelock}) \\
&\mid \mathrm{PlutusScript}(\mathrm{CardanoPlutusVersion}, \mathrm{PlutusBinary}) \\
\mathrm{CardanoPlutusVersion} &:= \mathrm{PlutusV1} \\
&\mid \mathrm{PlutusV2} \\
&\mid \mathrm{PlutusV3}
\end{aligned}
$$

1.5.2 Deviations from the Cardano transaction types

Midgard's transaction types deviate from Cardano's in the following ways:

No staking or governance actions. Midgard's consensus protocol is not based on Ouroboros proof-of-stake, and its governance protocol is not based on Cardano's hard-fork-combinator update mechanism. Furthermore, Midgard's L1 scripts cannot authorize arbitrary staking or governance actions on behalf of users. For this reason, staking and governance actions are prohibited in Midgard L2 transactions.

No pre-Conway features. Midgard does not need to maintain backwards compatibility with pre-Conway eras. Midgard deposits and transactions will be prohibited from using bootstrap addresses, and Shelley addresses with Plutus versions older than Plutus V3. Public key hash credentials, native scripts, and Plutus scripts at or above V3 are allowed.

No transaction metadata. Midgard prohibits using metadata in L2 transactions but allows users to set the `auxiliary_data_hash` to any arbitrary hash, which Midgard's ledger's rules do not require to match the empty transaction metadata field. This preserves users' ability to pin metadata content via a hash in Midgard's ledger, but they are expected to store the actual metadata offchain.

No datum hashes in outputs. Midgard requires all utxo datums to be inline, which avoids the need to index the datum-hash to datum map from transactions' datum witnesses.

CIPs 112 and 128. Midgard has adopted CIP-112 and CIP 128 ahead of Cardano mainnet, which we expect to merge them in 2025. CIP-112 provides a new "Observe" script purpose, a more principled and streamlined alternative to the "Withdraw 0" trick widely used by most Cardano dApps. It also allows native scripts to conditionally forward their logic to an observer script. On the other hand, CIP-128 will significantly improve the execution efficiency achievable in Plutus contracts by preserving the order of inputs in a submitted transaction (instead of ordering them by `OutputRef`).

Different network ID. Midgard transactions and utxo addresses use a different network ID to distinguish them from their Cardano mainnet counterparts. Furthermore, we are considering a prefix to minting policy IDs to distinguish tokens deposited from L1 from tokens minted on L2.

Replace slots with POSIX timestamps. Midgard's consensus protocol does not use slots like Ouroboros. Furthermore, the L1 contracts that enforce Midgard's consensus protocol are evaluated in the Plutus script context, where slots are converted to Posix timestamps. For these reasons, Midgard uses POSIX timestamps instead of slots in L2 transactions.

IsValid tag is always True. Midgard's consensus protocol does not use the IsValid tag, so it is always set to `True`.

1.5.3 Midgard simplified transaction types

Midgard's actual transaction types are complicated by the need to optimize their traversal by Plutus scripts. As an intermediate step towards them, the following simplified types illustrate how the Cardano transaction types (Section 1.5.1) are modified by Midgard's deviations (Section 1.5.2). We use the empty set symbol $\varnothing$ to indicate fields required to be empty in Midgard transactions and use the star symbol $\star$ to indicate new or modified fields:

$$\mathsf{MidgardSTx} := \left\{ \begin{array}{ll} \mathsf{body:} & \mathsf{MidgardSTxBody} \\ \mathsf{wits:} & \mathsf{MidgardSTxWits} \\ \mathsf{is\_valid:} & \mathsf{Bool} \\ \varnothing\ \mathsf{auxiliary\_data:} & \quad ?\ \mathsf{TxMetadata} \end{array} \right\}$$

$$
\mathsf{MidgardSTxBody} := \left\{\begin{array}{ll}
\mathsf{spend\_inputs}: & \mathsf{Set}(\mathsf{OutputRef}) \\
\varnothing\ \mathsf{collateral\_inputs}: & ?\ \mathsf{Set}(\mathsf{OutputRef}) \\
\mathsf{reference\_inputs}: & ?\ \mathsf{Set}(\mathsf{OutputRef}) \\
\mathsf{outputs}: & [\mathsf{Output}] \\
\varnothing\ \mathsf{collateral\_return}: & ?\ \mathsf{Output} \\
\varnothing\ \mathsf{total\_collateral}: & ?\ \mathsf{Coin} \\
\varnothing\ \mathsf{certificates}: & ?\ [\mathsf{Set}(\mathsf{Certificate})] \\
\varnothing\ \mathsf{withdrawals}: & ?\ \mathsf{Map}(\mathsf{RewardAccount}, \mathsf{Coin}) \\
\mathsf{fee}: & \mathsf{Coin} \\
\star\ \mathsf{validity\_interval}: & ?\ \mathsf{MidgardSValidityInterval} \\
\star\ \mathsf{required\_observers}: & ?\ [\mathsf{ScriptCredential}] \\
\mathsf{required\_signer\_hashes}: & ?\ [\mathsf{VKeyCredential}] \\
\mathsf{mint}: & ?\ \mathsf{Value} \\
\mathsf{script\_integrity\_hash}: & ?\ \mathsf{Hash} \\
\mathsf{auxiliary\_data\_hash}: & ?\ \mathsf{Hash} \\
\mathsf{network\_id}: & ?\ \mathsf{Network} \\
\varnothing\ \mathsf{voting\_procedures}: & ?\ \mathsf{VotingProcedures} \\
\varnothing\ \mathsf{proposal\_procedures}: & ?\ \mathsf{Set}(\mathsf{ProposalProcedure}) \\
\varnothing\ \mathsf{current\_treasury\_value}: & ?\ \mathsf{Coin} \\
\varnothing\ \mathsf{treasury\_donation}: & ?\ \mathsf{Coin}
\end{array}\right\}
$$

$$
\mathsf{MidgardSTxWits} := \left\{\begin{array}{ll}
\mathsf{addr\_tx\_wits}: & ?\ \mathsf{Set}(\mathsf{VKey}, \mathsf{Signature}, \mathsf{VKeyHash}) \\
\varnothing\ \mathsf{boot\_addr\_tx\_wits}: & ?\ \mathsf{Set}(\mathsf{BootstrapWitness}) \\
\mathsf{script\_tx\_wits}: & ?\ \mathsf{Map}(\mathsf{ScriptHash}, \mathsf{MidgardSScript}) \\
\varnothing\ \mathsf{data\_tx\_wits}: & ?\ \mathsf{TxDats} \\
\mathsf{redeemer\_tx\_wits}: & ?\ \mathsf{Redeemers}
\end{array}\right\}
$$

$$
\begin{aligned}
\mathsf{MidgardSValidityInterval} &:= (\mathsf{Option}(\mathsf{PosixTime}), \mathsf{Option}(\mathsf{PosixTime})) \\
\mathsf{MidgardSScript} &:= \mathsf{TimelockScript}(\star\ \mathsf{TimelockObserver}) \\
&\mid \mathsf{PlutusScript}(\mathsf{MidgardSPlutusVersion}, \mathsf{PlutusBinary}) \\
\mathsf{MidgardSPlutusVersion} &:= \mathsf{PlutusV3}
\end{aligned}
$$

1.5.4 Midgard transaction types

Midgard's transaction types modify the above simplified types by replacing all variable-length fields with hashes (indicated by the letter $\mathcal{H}$ below). The data availability layer is responsible for confirming that the hashes correspond to their preimages, and DA fraud proofs can be submitted if this correspondence is violated.

$$
\mathsf{MidgardTx} := \left\{\begin{array}{ll}
\mathsf{body}: & \mathcal{H}(\mathsf{MidgardTxBody}) \\
\mathsf{wits}: & \mathcal{H}(\mathsf{MidgardTxWits}) \\
\mathsf{is\_valid}: & \mathsf{Bool}
\end{array}\right\}
$$

$$
\text{MidgardTxBody} := \left\{ \begin{array}{ll}
\text{spend\_inputs}: & \mathcal{H}([\text{OutputRef}]) \\
\text{reference\_inputs}: & ?\,\mathcal{H}([\text{OutputRef}]) \\
\text{outputs}: & \mathcal{H}([\text{Output}]) \\
\text{fee}: & \text{Coin} \\
\text{validity\_interval}: & ?\,\text{MidgardSValidityInterval} \\
\text{required\_observers}: & ?\,\mathcal{H}([\text{ScriptCredential}]) \\
\text{required\_signer\_hashes}: & ?\,\mathcal{H}([\text{VKeyCredential}]) \\
\text{mint}: & ?\,\text{Value} \\
\text{script\_integrity\_hash}: & ?\,\text{Hash} \\
\text{auxiliary\_data\_hash}: & ?\,\text{Hash} \\
\text{network\_id}: & ?\,\text{Network}
\end{array} \right\}
$$

$$
\text{MidgardTxWits} := \left\{ \begin{array}{ll}
\text{addr\_tx\_wits}: & ?\,\mathcal{H}(\text{Set}(\text{VKey}, \text{Signature}, \text{VKeyHash})) \\
\text{script\_tx\_wits}: & ?\,\mathcal{H}(\text{Map}(\text{ScriptHash}, \text{MidgardSScript})) \\
\text{redeemer\_tx\_wits}: & ?\,\mathcal{H}(\text{Redeemers})
\end{array} \right\}
$$

1.6 Confirmed state

When a Midgard block becomes confirmed, a selection of its block header fields is split into two groups (discarding the rest). These groups are used to populate the fields of two record types in Midgard's confirmed state:

$$
\text{ConfirmedState} := \left\{ \begin{array}{ll}
\text{header\_hash}: & \text{HeaderHash} \\
\text{prev\_header\_hash}: & \text{HeaderHash} \\
\text{utxo\_root}: & \text{MPTR} \\
\text{start\_time}: & \text{PosixTime} \\
\text{end\_time}: & \text{PosixTime} \\
\text{protocol\_version}: & \text{Int}
\end{array} \right\}
$$

$$
\text{Settlement} := \left\{ \begin{array}{ll}
\text{deposit\_root}: & \text{MPTR} \\
\text{withdraw\_root}: & \text{MPTR} \\
\text{start\_time}: & \text{PosixTime} \\
\text{end\_time}: & \text{PosixTime}
\end{array} \right\}
$$

At genesis, `ConfirmedState` is set as follows:

$$
\text{genesisConfirmedState} := \left\{ \begin{array}{ll}
\text{header\_hash} := & 0 \\
\text{prev\_header\_hash} := & 0 \\
\text{utxo\_root} := & \text{MPTR}_{\text{empty}} \\
\text{start\_time} := & \text{system\_start} \\
\text{end\_time} := & \text{system\_start} \\
\text{protocol\_version} := & 0
\end{array} \right\}
$$

Midgard only stores the latest confirmed block's `ConfirmedState` on L1, always overwriting the previous one. By contrast, its `Settlement` never overwrites that of the previous block. Instead, the `Settlement` spins into a separate settlement node that users and operators can reference to process deposits and withdrawal requests.

When all deposits and withdrawals in a settlement node have been processed, the current operator (see Section 3.3) can mark it as optimistically resolved. It stays in this state for a duration set according to Midgard's `settlement_resolution_duration` protocol parameter, providing an opportunity for fraud proofs to be verified on L1 that disprove the operator's claim that the settlement node is resolved. Afterward, the settlement node is resolved, and the operator can spend it to recover its min-ADA.

> We're also considering an alternative model where confirmed block headers are not condensed on L1. Instead, they become immutable after confirmation, and we use a pointer to indicate the latest confirmed block header. This has broad implications for Midgard's onchain and offchain architecture.
> The advantage of this approach is that block headers are always available for scripts (including fraud proofs) to access directly on L1. This may simplify the data stored in each block.
> The disadvantage is that operators cannot recover the min-ADA deposit for every confirmed block. Furthermore, Midgard increasingly bloats Cardano's utxo set with utxos that are rarely accessed.

Chapter 2

User event protocol

The user event protocol controls how users interact with Midgard. It consists of three L1-based interactions and one L2-based interaction.

2.1 Deposit (L1)

A user deposits funds into Midgard by submitting an L1 transaction that performs the following:

1. Spend an input `l1_nonce`, which uniquely identifies this deposit transaction.
2. Register a staking script credential to witness the deposit. The script is parametrized by `l1_nonce`, and the credential's purpose is to disprove the existence of the deposit whenever the credential is *not* registered.[1]
3. Mint a deposit auth token to verify the following datum:

$$\mathsf{DepositDatum} := \left\{ \begin{array}{ll} \mathsf{event}: & \mathsf{DepositEvent}, \\ \mathsf{inclusion\_time}: & \mathsf{PosixTime}, \\ \mathsf{witness}: & \mathsf{ScriptHash}, \\ \mathsf{refund\_address}: & \mathsf{Address}, \\ \mathsf{refund\_datum}: & \mathsf{Option}(\mathsf{Data}) \end{array} \right\}$$

4. Send the user's deposited funds to the Midgard deposit address, along with the deposit auth token and the above datum.

At the time of the L1 deposit transaction, the deposit's `inclusion_time` is set to the sum of the transaction's validity interval upper bound and the `event_wait_duration` Midgard protocol parameter. According to Midgard's ledger rules:

Deposit inclusion. A block header must include deposit events with inclusion times falling within the block header's event interval, and it must *not* include any other deposit events.

[1] Cardano's ledger lacks a more direct method to disprove the existence of a utxo to a Plutus script.

Furthermore, the state queue enforces that event intervals are adjacent and non-overlapping. Therefore, while operators continue committing valid block headers, every deposit is eventually included in the state queue.

On the other hand, suppose a fraud proof is verified on L1 to prove that a block header in the state queue has violated the deposit inclusion rule. When this block header and all its descendants are removed, the state queue enforces that the next committed block header's event interval must contain all of those removed block headers' event intervals. Therefore, Midgard's ledger rules require this new block header to include all deposit events that should have been included in the removed block headers.

The deposit's outcome is determined as follows:

- If the deposit event is included in a settlement queue node, then the deposit utxo can be absorbed into the Midgard reserves or used to pay for withdrawals.
- If the deposit's inclusion time is within the confirmed header's event interval but not within the event interval of any settlement queue node, then the deposit utxo can be refunded to its user according to the `refund_address` and `refund_datum` fields.

The deposit's `witness` staking credential must be deregistered when the deposit utxo is spent.

2.1.1 Staking script

2.1.2 Minting policy

2.1.3 Spending validator

TODO

2.2 Transaction request (L2)

A user's primary method of transacting on the Midgard ledger is to submit an L2 transaction directly to the current operator via a web-based API endpoint. Operators are expected to serve such APIs in a publicly accessible and employ sufficient security techniques to mitigate denial-of-service.

Users' L2 transactions follow the data structure defined in Section 1.5.4. In particular, users can freely set transaction time-validity intervals according to their preferences. As long as there is a non-empty overlap between a transaction's time-validity interval and a block's event interval, the transaction can be included in the block. Since event intervals are adjacent to each other, a user's valid transaction request can only miss the ledger in two cases:

- The operator censors it (see mitigation in Section 2.3).
- The operator commits the last block overlapping with the transaction before receiving the transaction request.[2]

By contrast, L1 transactions on Cardano can fail in several other time-related ways. Ouroboros blocks only occupy every 20th one-second slot on average and have limited transaction capacity. This means that users often have to set longer transaction validity intervals than they want and wait for many block confirmations of their tx before relying on its outcomes.

However, while on Cardano L1 two transactions with non-overlapping validity intervals cannot be included in the same block, the analogous L2 transactions *can* be included in a Midgard block if its event interval overlaps each transaction. Thus, transaction validity intervals on Midgard define only the relation between transactions and blocks but do not necessarily imply a temporal precedence relation between transactions.

> Which other information should we include here for L2 transaction requests?

TODO

[2] Note that this case does not preclude transactions with short time-validity intervals from succeeding. For example, a transaction post-dated a couple of minutes in the future can still be included in a block even if its validity interval's duration is one millisecond.

2.3 Transaction order (L1)

A user who wants to mitigate the risk of censorship by the current operator can submit an L2 transaction as an L1 transaction order. A transaction order is created by an L1 transaction that performs the following:

1. Spend an input `l1_nonce`, which uniquely identifies this transaction order.
2. Register a staking script credential to witness the transaction order. The staking script is parametrized by `l1_nonce`, and the credential's purpose is to disprove the existence of the transaction order whenever the credential is *not* registered.
3. Mint a transaction order token to verify the following datum:

$$\mathsf{TxOrderDatum} := \left\{ \begin{array}{ll} \mathsf{tx}: & \mathsf{MidgardTx}, \\ \mathsf{inclusion\_time}: & \mathsf{PosixTime}, \\ \mathsf{witness}: & \mathsf{ScriptHash}, \\ \mathsf{refund\_address}: & \mathsf{Address}, \\ \mathsf{refund\_datum}: & \mathsf{Option}(\mathsf{Data}) \end{array} \right\}$$

4. Send min-ADA to the Midgard transaction order address, along with the transaction order token and the above datum.

At the time of the L1 transaction order, its `inclusion_time` is set to the sum of the L1 transaction's validity interval upper bound and the `event_wait_duration` Midgard protocol parameter. According to Midgard's ledger rules:

Transaction order inclusion. A block header must include transaction orders with inclusion times falling within the block header's event interval, and it must *not* include any other transaction orders.

Analogously to deposits, transaction orders will eventually be included in the state queue, as long as operators continue committing valid block headers. Furthermore, if any blocks are removed from the state queue, any new committed block must include the transaction orders that should have been included in the removed blocks.

The transaction order fulfills its purpose when its inclusion time is within the confirmed header's event interval. Whether or not the outcome of the order's `tx` L2 transaction was merged into the confirmed state, nothing more can be achieved with the transaction order, and it can be refunded according to the `refund_address` and `refund_datum`.

The transaction order's `witness` staking credential must be deregistered when the deposit utxo is spent.

2.3.1 Staking script

2.3.2 Minting policy

> The transaction order's `inclusion_time` must be within its `tx` L2 transaction's time-validity interval.

2.3.3 Spending validator

TODO

2.4 Withdrawal order (L1)

Currently, this withdrawal order design can only handle withdrawing pubkey-based utxos. However, we can modify L2 transactions to produce "withdrawable utxos" that cannot be spent on L2 but can be withdrawn permissionlessly on L1. This will allow L1 withdrawal orders to withdraw these L2 utxos without even requiring any signatures.

A user initiates a withdrawal from Midgard by submitting an L1 transaction that performs the following:

1. Spend an input `l1_nonce`, which uniquely identifies this withdrawal order.
2. Register a staking script credential to witness the withdrawal order. The staking script is parametrized by `l1_nonce`, and the credential's purpose is to disprove the existence of the withdrawal order whenever the credential is *not* registered.
3. Mint a withdrawal order token to verify the following datum:

$$\text{WithdrawalOrderDatum} := \left\{ \begin{array}{ll} \text{event}: & \text{WithdrawalEvent}, \\ \text{inclusion\_time}: & \text{PosixTime}, \\ \text{witness}: & \text{ScriptHash}, \\ \text{refund\_address}: & \text{Address}, \\ \text{refund\_datum}: & \text{Option}(\text{Data}) \end{array} \right\}$$

4. Send min-ADA to the Midgard withdrawal order address, along with the withdrawal order token and the above datum.

At the time of the L1 withdrawal order, its `inclusion_time` is set to the sum of the L1 transaction's validity interval upper bound and the `event_wait_duration` Midgard protocol parameter. According to Midgard's ledger rules:

Withdrawal order inclusion. A block header must include withdrawal orders with inclusion times falling within the block header's event interval, and it must *not* include any other withdrawal orders.

The withdrawal order's outcome is determined as follows:

- If the withdrawal event is included in a settlement queue node, then utxos from the Midgard reserve and confirmed deposits can be used to pay for the creation of an L1 utxo according to the withdrawal event.
- If the withdrawal order's inclusion time is within the confirmed header's event interval but not within the event interval of any settlement queue node, then the withdrawal order utxo can be refunded to its user according to the `refund_address` and `refund_datum` fields.

The withdrawal order's `witness` staking credential must be deregistered when the withdrawal order utxo is spent.

2.4.1 Staking script

2.4.2 Minting policy

> The transaction must be signed by the pubkey corresponding to the withdrawal event's `l2_outref` payment credential.

2.4.3 Spending validator

TODO

Chapter 3

Consensus protocol

This chapter describes Midgard's L1 contract-based consensus protocol, which establishes the canonical chain of valid blocks. It consists of the following components:

- The operator directory is an onchain data structure that tracks active, retired, and newly registered Midgard operators.
- The scheduler is an onchain mechanism that assigns evenly sized time windows to operators on a rotating schedule.
- The state queue is an onchain data structure that holds operators' committed block headers until they are merged into the confirmed state or disqualified by fraud proofs.
- When operators aren't committing blocks for a long time, the escape-hatch mechanism can be used to commit a non-optimistic block that includes transaction and withdrawal orders for which full compliance proofs have been verified on L1.
- Computation threads facilitate the onchain validation of submitted fraud proofs, splitting it up into steps that fit within Cardano's limits on transaction size, computation, and memory.
- The fraud proof catalogue defines the universe of fraud proof verification procedures for which computation threads can be spawned to target a block header in the state queue.
- A fraud proof token indicates that a computation thread has successfully concluded to verify a fraud proof about a block header in the state queue.
- The Midgard hub oracle wires all the onchain components together by storing their minting policy IDs and spending validator addresses for easy reference.

3.1 Time model

Midgard partitions time in two different ways:

Operator shifts. Predefined, non-overlapping time intervals assigned to operators by the Midgard scheduler. Each operator has the exclusive privilege to commit blocks to the state queue and resolve nodes in the settlement queue during their assigned shifts. If an operator fails to commit blocks regularly in their shift, the next operator can take over.

Event intervals. Emergent, non-overlapping time intervals claimed by operators' committed blocks. Each operator block is expected to include all user events with inclusion times within its event interval and to exclude all other user events. For L1 user events (deposits, transaction orders, withdrawal orders), this is enforced by Midgard's ledger rules.

Operator shifts are evenly sized according to the `shift_duration` Midgard protocol parameter. The scheduler assigns operators to shifts by iterating over the list of active operators in key-descending order, allowing new operators into the list at the end of each cycle. The state queue enforces this schedule by requiring the time-validity upper bound of every block commitment transaction to fall within its operator's shift.

Each block's event interval is between the previous block's time-validity upper bound and its own time-validity upper bound. The block's time-validity lower bound is irrelevant to Midgard's consensus protocol because the causal relationship between blocks is determined by the forward links between state queue nodes, as well as the backwards links between block headers. An operator is free to submit blocks at a rapid cadence during their shift.[1]

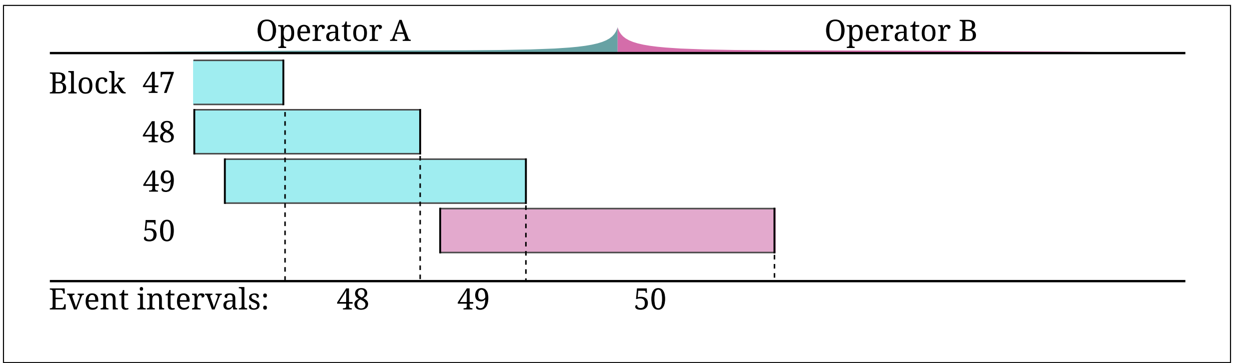

Figure 3.1: Midgard time model. Each rectangle shows a block's time-validity interval. The color indicates the operator who must have committed the block. The top axis shows the operator shifts. The bottom axis shows the event intervals.

[1] Indeed, an operator may commit blocks without waiting for L1 confirmation. As long as the blocks are valid, no one can interfere with the operator's chain of block commitment transactions. Moreover, if operators coordinate their activity offchain, this rapid cadence can be mostly maintained across shift boundaries.

3.2 Operator directory

Midgard operators receive and process L2 events from users, collate them into blocks, publish the full block contents on the data availability layer, and commit the block headers to Midgard's state queue on L1. This responsibility is shared among the operators on a rotating schedule, with each operator getting a turn to exclusively process L2 events and then commit their block of events at the end of their turn.

Anyone can register to become a Midgard operator if they post the required ADA bond. Registrants must wait a prescribed period of time before they activate as an operator. An active operator can retire to remove themself from the rotating schedule. Retired operators must wait until all their committed block headers mature before recovering their ADA bond.

An operator's ADA bond collateralizes their promise to faithfully process L2 events and commit valid blocks to L1. If an operator's block header is proven to be fraudulent, then the operator is disqualified and forfeits their bond. Similarly, an operator that submits duplicate registrations forfeits the bonds placed in the duplicates.

Every forfeited bond is split into a reward paid to the fraud prover and a slashing penalty paid (via transaction fees) to the Cardano treasury. Among the Midgard protocol parameters, the `fraud_prover_reward` and `slashing_penalty` parameters must sum up to the `required_bond` parameter.

3.2.1 Utxo representation

The Operator Directory keeps track of all Midgard operators and stores their bond deposits. It separates operators into three groups based on their status, with every group using operators' public key hashes (PKHs) as keys:

- `registered_operators` is a first-in-first-out (FIFO) queue that tracks operators who have posted their ADA bonds and are waiting to activate. It is implemented as a key-unordered linked list (see Appendix A.1.8), where each new registrant is prepended to the beginning of the list. This means that the earliest registrant to become eligible for activation is always at the last node of the list.
- `active_operators` is a set that tracks active operators participating in the rotating schedule, implemented as a key-ordered linked list (see Appendix A.1.9).
- `retired_operators` is a set that tracks retired operators waiting to recover their ADA bonds, implemented as a key-ordered linked list.

The Operator Directory keeps track of the registration time of every registered operator. It also keeps track of the latest commitment times of every active and retired operator so that retired operators cannot recover their ADA bonds before their latest committed block headers mature.

$$\begin{aligned}
\mathsf{RegisteredOperator} &:= \left(\mathsf{registration_time} : \mathsf{PosixTime}\right) \\
\mathsf{ActiveOperator} &:= \left(\mathsf{commitment_time} : \mathsf{Option}(\mathsf{PosixTime})\right) \\
\mathsf{RetiredOperator} &:= \left(\mathsf{commitment_time} : \mathsf{Option}(\mathsf{PosixTime})\right)
\end{aligned}$$

The above are app-specific types for the `data` field of their respective lists' `NodeDatum`. For example, the `NodeDatum` of `registered_operators` is:

$$\text{RegisteredOperatorDatum} \coloneqq \text{NodeDatum}(\text{RegisteredOperator})$$

3.2.2 Registered operators

The `registered_operators` queue keeps track of operators after registering and before activating or de-registering them.

Minting policy

The `registered_operators` minting policy implements state transitions for its key-unordered linked list. It is statically parametrized on the `hub_oracle` and `retired_operators` minting policies. Redeemers:

Init. Initialize the `registered_operators` queue via the Midgard hub oracle. Conditions:

1. The transaction must mint the Midgard hub oracle NFT.
2. The transaction must Init the `registered_operators` queue.

Deinit. Deinitialize the `registered_operators` queue via the Midgard hub oracle. Conditions:

1. The transaction must burn the Midgard hub oracle NFT.
2. The transaction must Deinit the `registered_operators` queue.

Register Operator. Prepend a node into the `registered_operators` queue with the operator's ADA bond at the operator's key, noting the registration time. Grouped conditions:

- Verify the registration and prepend the registrant to the registered operators:
  1. The transaction must Prepend a node into the `registered_operators` queue. Let that node be `registered_node`.
  2. The `key` field of `registered_node` must correspond to a public key hash that signed the transaction.
  3. The `registration_time` field of `registered_node` must equal the upper bound of the transaction's validity interval.
  4. The Lovelace in the `registered_node` must equal the `required_bond` Midgard parameter.
- Verify that the registrant is not already active or retired.
  5. The transaction must include the Midgard hub oracle NFT in a reference input.
  6. Let `active_operators` be the policy ID in the corresponding field of the Midgard hub oracle.
  7. The transaction must include a reference input of a `active_operators` node that proves the `registered_node` key's non-membership in that list.

8. The transaction must include a reference input of a `retired_operators` node that proves the `registered_node` key's non-membership in that list.

Activate Operator. Transfer an operator node from the `registered_operators` queue to the `active_operators` set. Grouped conditions:

- Remove the registrant from the registered operators if it is eligible to activate:

1. The transaction must Remove a node from the `registered_operators` queue. Let that node be `registered_node`.
2. The lower bound of the transaction validity interval must meet or exceed the sum of the `registration_time` and the Midgard `registration_duration` protocol parameter.

- Verify that the registrant is being added to active operators:

3. The transaction must include the Midgard hub oracle NFT in a reference input.
4. Let `active_operators` be the policy ID in the corresponding field of the Midgard hub oracle.
5. The transaction must Insert a node into the `active_operators` set, as evidenced by minting a `active_operators` node NFT for the `registered_node` key.

- Verify that the registrant is not already retired:

6. The transaction must include a reference input of a `retired_operators` node that proves the `registered_node` key's non-membership in that list.

Deregister Operator. Remove a node from the `registered_operators` queue, with the operator's consent, and return the ADA bond to the operator. Conditions:

1. The transaction must Remove a node from the `registered_operators` queue. Let `registered_node` be that node.
2. The `key` field of `registered_node` must correspond to a public key hash that signed the transaction.

The operator consents to the transaction, so the ADA bond is assumed to be returned to the operator's control.

Remove Duplicate Slash Bond. Remove a node from the `registered_operators` queue if its key duplicates the key of any other node among the registered, active, or retired operators. Do *not* return the duplicate node's ADA bond to its operator. Conditions:

1. The transaction must Remove a node from the `registered_operators` queue. Let that node be `duplicate_node`.
2. The transaction fees must meet or exceed the `slashing_penalty` protocol parameter, denominated in Lovelaces.
3. Let `witness_status` be one of: `Registered`, `Active`, or `Retired`.
4. If `witness_status` is `Registered`:

(a) The transaction must include a reference input of a registered_operators node that proves the duplicate_node key's membership in that list.

5. If witness_status is Active:

    (a) The transaction must include the Midgard hub oracle NFT in a reference input.

    (b) Let active_operators be the policy ID in the corresponding field of the Midgard hub oracle.

    (c) The transaction must include a reference input of a active_operators node that proves the duplicate_node key's membership in that list.

6. If witness_status is Retired:

    (a) The transaction must include a reference input of a retired_operators node that proves the duplicate_node key's membership in that list.

The submitter of the Remove Duplicate Slash Bond transaction is considered to be the fraud prover, so the conditions for that redeemer do not need to explicitly enforce that the fraud_prover_reward is paid out because the submitter consents to the transaction.

Spending validator

The spending validator of registered_operators_addr always forwards to its corresponding minting policy (statically parametrized) and requires the transaction to invoke it. It does not allow any in-place modifications to the RegisteredOperator value of the node data field. Conditions:

1. The transaction must mint or burn tokens of the registered_operators minting policy.

3.2.3 Active operators

The active_operators set keeps track of operators after activating and before slashing or retiring them.

Minting policy

The active_operators minting policy implements state transitions for its key-ordered linked list. It is statically parametrized on the hub_oracle, registered_operators, and retired_operators minting policies. Redeemers:

Init. Initialize the active_operators set via the Midgard hub oracle. Conditions:

1. The transaction must mint the Midgard hub oracle NFT.
2. The transaction must Init the active_operators set.

Deinit. Deinitialize the active_operators set via the Midgard hub oracle. Conditions:

1. The transaction must burn the Midgard hub oracle NFT.
2. The transaction must Deinit the active_operators set.

Activate Operator. Transfer an operator node from the registered_operators queue to the active_operators set. Conditions:

1. The transaction must Insert a node into the `active_operators` set. Let that node be `active_node`.
2. The `commitment_time` field of `active_node` must be `None`.
3. The transaction must Remove a node from the `registered_operators` queue, as evidenced by burning a `registered_operators` node NFT corresponding to the `active_node` key.

Remove Operator Slash Bond. Remove an operator's node from the `active_operators` set without returning the operator's ADA bond to the operator. Conditions:

1. Let `slashed_operator` be a redeemer argument indicating the operator being slashed.
2. The transaction must Remove a node from the `active_operators` set. Let that node be `removed_node`.
3. `slashed_operator` must match the key of `removed_node`.
4. The transaction fees must meet or exceed the `slashing_penalty` protocol parameter, denominated in Lovelaces.
5. The transaction must include the Midgard hub oracle NFT in a reference input.
6. Let `state_queue` be the policy ID in the corresponding field of the Midgard hub oracle.
7. The transaction must Remove a node from the `state_queue` via the Remove Fraudulent Block Header redeemer. The `fraudulent_operator` argument provided to that redeemer must match `slashed_operator`.

The state queue's onchain code is responsible for disposing of the operator's ADA bond.

Retire Operator. Transfer an operator node, unchanged, from the `active_operators` set to the `retired_operators` set. Conditions:

1. The transaction must Remove a node from the `active_operators` set. Let that node be `active_node`.
2. The transaction must Insert a node into the `retired_operators` set, as evidenced by minting a `retired_operators` node NFT corresponding to the `active_node` key.
3. Let `retired_node` be the node inserted into the `retired_operators` set.
4. The `commitment_time` must match between `active_node` and `retired_node`.

Spending validator

The spending validator of `active_operators_addr` forwards to its corresponding minting policy (statically parametrized) when the transaction invokes it. When the policy isn't invoked, the spending validator allows an operator's commitment time to be updated. Redeemers:

List State Transition. Forward to minting policy. Conditions:

1. The transaction must mint or burn tokens of the `active_operators` minting policy.

Update Commitment Time. Update an operator's commitment time when they commit a block to the state queue. Grouped conditions:

- Update the commitment time of a operator:
  1. The transaction must *not* mint or burn tokens of the `active_operators` minting policy.
  2. Let `active_node` be an output of the transaction indicated by a redeemer argument.
  3. `active_node` must be an `active_operators` node that matches the datum argument of the spending validator on the `key` and `link` fields.
  4. The `commitment_time` field of `active_node` must match the upper bound of the transaction validity interval.
- Verify that the operator is currently committing a block header to the state queue:
  5. The transaction must include the Midgard hub oracle NFT in a reference input.
  6. Let `state_queue` be the policy ID in the corresponding field of the Midgard hub oracle.
  7. The transaction must Append a node into the `state_queue` via the Commit Block Header redeemer. The redeemer's `operator` field must match the `key` field of the `active_node`.

3.2.4 Retired operators

The `retired_operators` set keeps track of operators after retiring and before slashing or returning their ADA bonds.

Minting policy

The `retired_operators` minting policy implements structural operations for its key-ordered linked list. It is statically parametrized on the `hub_oracle` minting policy. Redeemers:

Init. Initialize the `retired_operators` set via the Midgard hub oracle. Conditions:

1. The transaction must mint the Midgard hub oracle NFT.
2. The transaction must Init the `retired_operators` set.

Deinit. Deinitialize the `retired_operators` set via the Midgard hub oracle. Conditions:

1. The transaction must burn the Midgard hub oracle NFT.

2. The transaction must Deinit the `retired_operators` set.

Retire Operator. Transfer an operator node, unchanged, from the `active_operators` set to the `retired_operators` set. Conditions:

1. The transaction must Insert a node into the `retired_operators` set. Let that node be `retired_node`.
2. The transaction must include the Midgard hub oracle NFT in a reference input.
3. Let `active_operators` be the policy ID in the corresponding field of the Midgard hub oracle.
4. The transaction must Remove a node from the `active_operators` set, as evidence by the burning of a `active_operators` node NFT corresponding to the `retired_node` key.

The active operators' minting policy ensures that the operator node's contents remain unchanged during the transfer.

Recover Operator Bond. Remove an operator's node from the `retired_operators` set, with the operator's consent, and return the ADA bond to the operator. Grouped conditions:

1. The transaction must Remove a node from the `retired_operators` set. Let that node be `retired_node`.
2. If the `commitment_time` field of `retired_node` is *not* `None`, then the lower bound of the transaction validity interval must meet or exceed the sum of `commitment_time` and the Midgard `maturity_duration` protocol parameter.

The operator consents to the transaction, so the ADA bond is assumed to be returned to the operator's control.

Remove Operator Slash Bond. Remove an operator's node from the `retired_operators` set without returning the operator's ADA bond to the operator. Conditions:

1. Let `slashed_operator` be a redeemer argument indicating the operator being slashed.
2. The transaction must Remove a node from the `retired_operators` set. Let that node be `removed_node`.
3. `slashed_operator` must match the key of `removed_node`.
4. The transaction fees must meet or exceed the `slashing_penalty` protocol parameter, denominated in Lovelaces.
5. The transaction must include the Midgard hub oracle NFT in a reference input.
6. Let `state_queue` be the policy ID in the corresponding field of the Midgard hub oracle.
7. The transaction must Remove a node from the `state_queue` via the Remove Fraudulent Block Header redeemer. The `fraudulent_operator` argument provided to that redeemer must match `slashed_operator`.

The state queue's onchain code is responsible for paying out the fraud prover's reward from the operator's forfeited ADA bond.

Spending validator

The spending validator of `retired_operators_addr` always forwards to its corresponding minting policy (statically parametrized) and requires the transaction to invoke it. It does not allow any in-place modifications to the `RegisteredOperator` value of the node `data` field. Conditions:

1. The transaction must mint or burn tokens of the `retired_operators` minting policy.

3.3 Scheduler

The Midgard scheduler is an L1 mechanism that indicates which operator is assigned to the current shift and controls the transitions to the next shift and next operator. Whenever a shift ends, the next operator in key-descending order from the `active_operators` list has the exclusive privilege to advance the scheduler's state to the next shift, assigning it to themself.

When the root node of `active_operators` is reached, the highest-key active operator rewinds the scheduler to start a new cycle. However, the scheduler requires the `registered_operators` queue to be checked to see if any registered operators are eligible to activate. If so, all eligible registered operators must activate before the new cycle can begin.

Active operators can retire anytime without waiting for the end of the scheduler cycle. The scheduler automatically adjusts to retirement because the next operator is always selected among active operators. However, to minimize disruption, an operator should complete their shift before retiring.

> We should allow operators to take over for negligent operators.
> If a shift's assigned operator neglects their duty to commit blocks regularly to the state queue, the next active operator can take over the shift (without starting the next shift). Similarly, the next operator can advance the scheduler and take over if their predecessor neglects to do so. Midgard's `operator_inactivity_timeout` protocol parameter controls when this unscheduled takeover can happen.

TODO

3.3.1 Utxo representation

The scheduler state consists of a single utxo that holds the scheduler NFT, minted when Midgard is initialized and burned when Midgard is deinitialized via the hub oracle. That utxo's datum type is as follows:

$$\text{SchedulerDatum} := \left\{ \begin{array}{ll} \text{operator}: & \text{PubKeyHash} \\ \text{shift\_start}: & \text{PosixTime} \end{array} \right\}$$

The shift's inclusive lower bound is `shift_start`, and its exclusive upper bound is the sum of `shift_start` and the `shift_duration` Midgard protocol parameter.

3.3.2 Minting policy

The `scheduler` minting policy initializes and deinitializes the scheduler state. It is statically parametrized on the `hub_oracle` minting policy. Redeemers:

Init. Initialize the `scheduler` via the Midgard hub oracle. Conditions:

1. The transaction must mint the Midgard hub oracle token.
2. The transaction must mint the `scheduler` NFT.

Deinit. Deinitialize the `scheduler` via the Midgard hub oracle. Conditions:

1. The transaction must burn the Midgard hub oracle token.
2. The transaction must burn the `scheduler` NFT.

3.3.3 Spending validator

The spending validator of `scheduler_addr` controls the evolution of the scheduler state. It is statically parametrized on the `active_operators` and `registered_operators` minting policies. Redeemers:

Advance. The next operator in key-descending order advances the scheduler to the next shift and assigns it to themself. Grouped conditions:

- Parse scheduler input and output:
  1. Let `scheduler_datum` be the datum argument for the spending validator being evaluated. It is assumed to belong to the transaction input that holds the scheduler NFT.
  2. Let `scheduler_output` be the transaction output that holds the scheduler NFT.
  3. Let `operator` be the `operator` field value of `scheduler_output`.
  4. Let `previous_operator` be the `operator` field value of `scheduler_datum`.
- Verify the shift transition:
  5. The shift interval of `scheduler_output` must equal that of `scheduler_datum`, moved forward by `shift_duration`.
- Verify operator consent:
  6. Either of the following must hold:
     (a) The transaction is signed by `operator`, and the `scheduler_output` shift interval contains the transaction validity interval.
     (b) The `scheduler_output` shift interval occurs entirely before the transaction validity interval.
- Verify the operator transition:
  7. The transaction must include a reference input of an `active_operators` node. Let `operator_node` be that node.
  8. The `key` of `operator_node` must match `operator`.
  9. The `link` of `operator_node` must match or exceed `previous_operator`.[2]

Rewind. The highest-key operator advances the scheduler to the next shift and assigns it to themself, provided that no registered operators are eligible to activate. Grouped conditions:

- Parse scheduler input and output:
  1. Let `scheduler_datum` be the datum argument for the spending validator being evaluated. It is assumed to belong to the transaction input that holds the scheduler NFT.
  2. Let `scheduler_output` be the transaction output that holds the scheduler NFT.

[2] It will exceed the previous operator if they have retired.

3. Let `operator` be the `operator` field value of `scheduler_output`.
4. Let `previous_operator` be the `operator` field value of `scheduler_datum`.

- Verify the shift transition:

5. The shift interval of `scheduler_output` must equal that of `scheduler_datum`, moved forward by `shift_duration`.

- Verify operator consent:

6. Either of the following must hold:
   (a) The transaction is signed by `operator`, and the `scheduler_output` shift interval contains the transaction validity interval.
   (b) The `scheduler_output` shift interval occurs entirely before the transaction validity interval.

- Rewind to a new operator cycle:

7. The transaction must include a reference input of the last `active_operators` node. Let `operator_node` be that node.
8. The transaction must include a reference input of the root `active_operators` node. Let `root_node` be that node.
9. The `key` of `operator_node` must match `operator`.
10. The `link` of `root_node` must match or exceed `previous_operator`.

- Verify that no registered operator is eligible to activate:

11. The transaction must include a reference input of a `registered_operators` node. Let `registered_node` be that node.
12. `registered_node` must be the last node of the `registered_operators` queue. This means it corresponds to the *earliest* registrant that hasn't yet activated because new registrants are prepended to the beginning of that queue.
13. `registered_node` is *not yet* eligible for activation — the lower bound of the transaction validity interval is smaller than the sum of the `registration_time` of the `registered_node` and the Midgard `registration_duration` protocol parameter.

3.4 State queue

The state queue is an L1 data structure that stores Midgard operators' committed block headers until they are confirmed. Active operators from the operator directory (see Section 3.2) take turns committing block headers to the state queue according to the rotating schedule enforced by the scheduler (see Section 3.3).

3.4.1 Utxo representation

The `state_queue` is implemented as a key-unordered linked list of block headers (see Section 1.1). Each `state_queue` node's key is its block header's hash.

$$\mathsf{StateQueueDatum} := \mathsf{NodeDatum}(\mathsf{Header})$$

$$\mathsf{valid\_key}(\mathsf{scd} : \mathsf{StateQueueDatum}) := \Big(\mathsf{scd.key} \equiv \mathsf{Some}(\mathsf{hash}(\mathsf{scd.data}))\Big)$$

Committing a block header to the `state_queue` means appending a node containing the block header to the end of the queue. After staying there for the `maturity_duration` (a protocol parameter), it is merged to the confirmed state (held at the `state_queue` root node) in the first-in-first-out (FIFO) order.

3.4.2 Minting policy

The `state_queue` minting policy controls the structural changes to the state queue. It is statically parametrized on the `hub_oracle`, `active_operators`, `retired_operators`, `scheduler`, and `fraud_proof` minting policies. Redeemers:

Init. Initialize the `state_queue` via the Midgard hub oracle. Conditions:

1. The transaction must mint the Midgard hub oracle token.
2. The transaction must Init the `state_queue`.

Deinit. Deinitialize the `state_queue` via the Midgard hub oracle. Conditions:

1. The transaction must burn the Midgard hub oracle token.
2. The transaction must Deinit the `state_queue`.

Commit Block Header. An operator commits a block header to the state queue if it is the operator's turn according to the rotating schedule. Grouped conditions:

- Commit the block header to the state queue, with the operator's consent:
  1. Let `operator` be a redeemer argument indicating the key of the operator committing the block header.
  2. The transaction must be signed by `operator`.
  3. The transaction must Append a node to the `state_queue`. Among the transaction outputs, let that node be `header_node` and its predecessor node be `previous_header_node`.

4. `operator` must be the operator of `header_node`.
5. Let `previous_operator` be the operator of `previous_header_node`.
6. The `key` field of `header_node` must be the hash of its `data` field.

- Verify that it is the operator's turn to commit according to the scheduler:

7. The transaction must include a reference input with the scheduler NFT. Let that input be the `scheduler_state`.
8. The `operator` field of `scheduler_state` must match `operator`.

- Verify block timestamps:

9. The `start_time` of `header_node` must be equal to the `end_time` of the `previous_header_node`.
10. The `end_time` of `header_node` must match the transaction's time-validity upper bound.
11. The `end_time` of `header_node` must be within the shift interval of `scheduler_state`.

- Update the operator's timestamp in the active operators set:

12. The transaction must include an input, spent via the Update Commitment Time redeemer, of an `active_operators` node with a key matching the `operator`. Let `operator_node` be that node.

Merge To Confirmed State. If a block header is mature, merge it to the confirmed state of the `state_queue`. Conditions:

1. The transaction must Remove a node from the `state_queue`. Let `header_node` be the removed node and `confirmed_state_node` be its predecessor node before removal.
2. `confirmed_state_node` must be the root node of `state_queue`.
3. `header_node` must be mature — the lower bound of the transaction validity interval meets or exceeds the sum of the `commitment_time` field of `header_node` and the Midgard `maturity_duration` protocol parameter.
4. `header_node` and `confirmed_state_node` must match on the `data` field.

Remove Fraudulent Block Header. Remove a fraudulent block header from the state queue and slash its operator's ADA bond. Grouped conditions:

- Remove the fraudulent block header:

1. Let `fraudulent_operator` be a redeemer argument indicating the operator who committed the fraudulent block header.
2. The transaction must Remove a node from the `state_queue`. Let `removed_node` be the removed node and `predecessor_node` be its predecessor node before removal.
3. `fraudulent_operator` must match the key of `removed_node`.

- Slash the fraudulent operator:

  4. Let operator_status be a redeemer argument indicating whether the fraudulent operator is active or retired.

  5. If operator_status is active:

     (a) The transaction must Remove a node from the active_operators set via the Remove Operator Slash Bond redeemer. The slashed_operator argument provided to that redeemer must match fraudulent_operator.

  6. Otherwise:

     (a) The transaction must Remove a node from the retired_operators set via the Remove Operator Slash Bond redeemer. The slashed_operator argument provided to that redeemer must match fraudulent_operator.

- Verify that fraud has been proved for the removed node or its predecessor:

  7. The transaction must include a reference input holding a fraud_proof token.

  8. Let fraud_proof_block_hash be the last 28 bytes of the fraud_proof token.

  9. One of the following must be true:

     (a) fraud_proof_block_hash matches the predecessor_node key. This means that a child of the fraudulent node is being removed.

     (b) fraud_proof_block_hash matches the removed_node key, and last node of state_queue is removed_node. This means that the fraudulent node is being removed and has no more children.

> Add redeemers for escape hatch blocks.

TODO

3.4.3 Spending validator

The spending validator of state_queue_addr always forwards to its corresponding minting policy (statically parametrized) and requires the transaction to invoke it. It does not allow any in-place modifications to the data field of nodes in state_queue. Conditions:

1. The transaction must mint or burn tokens of the state_queue minting policy.

3.5 Escape hatch

TODO

An escape hatch block can be added to the state queue as follows:

1. Spend transaction order and withdrawal order utxos, refunding all of their min-ADA.
2. Reference compliance proof tokens verifying that the spent orders fully comply with Midgard's ledger rules.
3. Append a node to the state queue with a special EscapeHatch datum.

If the escape hatch block is removed (because it is a child of a fraudulent block), its transaction and withdrawal orders are lost. However, they can be recreated, and their compliance proofs can be reused to create another escape hatch block (omitting any orders that are no longer valid).

An escape hatch block is merged into the confirmed state like an operator block: copy over the relevant fields to the ConfirmedHeader and append a node to the settlement queue if the escape hatch block contains any withdrawal events.

3.6 Settlement queue

A queue similar to the state queue. Nodes are appended to it for any state queue blocks that are merged into the confirmed state and contain deposit or withdrawal events.

Users refer to state queue nodes to spend the corresponding withdrawal order and deposit utxos, along with any needed reserve utxos, to execute the corresponding deposit and withdrawal events.

When all deposit and withdrawal events in a settlement node have been executed, the current operator can optimistically mark it as resolved. Once the maturity period elapses for this optimistic claim, the operator can remove the node from the settlement queue and claim fees from the Midgard reserve corresponding to the number of deposits and withdrawals contained in the node. Midgard's protocol is responsible for deducting those fees from the deposits and withdrawals accordingly.

The timestamp in the operator's node in the Operator Directory should be updated whenever an operator optimistically resolves a settlement node, in the same way that it is updated when committing a block to the state queue.

> Apply changes to the Operator directory (Section 3.2) to ensure that an operator's bond is not released to the operator until all last settlement nodes are resolved.

TODO

> Perhaps resolving a settlement node should also release to the operator the user fees collected for processing the deposits and withdrawals in the node. This would incentivize the operator to promptly resolve settlement nodes.

3.7 Reserve

When Midgard confirms deposit events, their funds are either transferred into Midgard's reserve or used directly to pay out confirmed withdrawals. Even when deposits are directly used to pay out withdrawals, any leftover funds from the deposits are transferred into Midgard's reserve.

Midgard's reserve smart contract is responsible for safeguarding these funds, ensuring that they can only be used to pay out confirmed withdrawals. It also allows the current operator to rearrange the funds within the Reserve spending validator into utxos that are more optimized to process withdrawals of varying sizes with minimal contention. **TODO**

3.8 Midgard hub oracle

This oracle keeps track of minting policy IDs and spending validator addresses for the lists used in the operator directory, state queue, and fraud proof set of the Midgard protocol. It consists of a single utxo holding the hub oracle NFT and a datum of the following type:

$$\text{HubOracleDatum} := \left\{ \begin{array}{ll} \text{registered\_operators}: & \text{PolicyId} \\ \text{active\_operators}: & \text{PolicyId} \\ \text{retired\_operators}: & \text{PolicyId} \\ \text{scheduler}: & \text{PolicyId} \\ \text{state\_queue}: & \text{PolicyId} \\ \text{fraud\_proof\_catalogue}: & \text{PolicyId} \\ \text{fraud\_proof}: & \text{PolicyId} \\ & \\ \text{registered\_operators\_addr}: & \text{Address} \\ \text{active\_operators\_addr}: & \text{Address} \\ \text{retired\_operators\_addr}: & \text{Address} \\ \text{scheduler\_addr}: & \text{Address} \\ \text{state\_queue\_addr}: & \text{Address} \\ \text{fraud\_proof\_catalogue\_addr}: & \text{Address} \\ \text{fraud\_proof\_addr}: & \text{Address} \end{array} \right\}$$

Add missing policy IDs and addresses to the hub oracle datum.

TODO

3.8.1 Minting policy

The `hub_oracle` minting policy ensures that all Midgard lists are initialized together, sent to their respective spending validator addresses, and deinitialized together. Redeemers:

Init. Initialize all Midgard lists and send their root nodes to their respective validator addresses. Conditions:

1. Let `nonce_utxo` be a static parameter of the `hub_oracle` minting policy.
2. `nonce_utxo` must be spent.
3. The hub oracle NFT must be minted.
4. Let `hub_oracle_output` be the transaction output with the hub oracle NFT.
5. `hub_oracle_output` must *not* contain any other non-ADA tokens.
6. `hub_oracle_output` must be sent to the `burn_everything` spending validator address.
7. The root node NFT of every linked list policy ID in `hub_oracle_output` must be minted and sent to the corresponding spending validator address in `hub_oracle`.
8. The NFT of the `scheduler` policy ID in `hub_oracle_output` must be minted and sent to the `scheduler_addr`.

9. No other tokens must be minted or burned.

The nonce utxo proves authority for initialization — whoever controls it is authorized to initialize the Midgard L1 data structures.

Burn. Deinitialize all Midgard lists and burn the hub oracle NFT. Conditions:

1. The transaction must burn the hub oracle NFT.
2. Let `hub_oracle_input` be the transaction input that holds the hub oracle NFT.
3. The root node NFT of every linked list policy ID in the `hub_oracle_input` datum must be burned.
4. The NFT of the `scheduler` policy ID in `hub_oracle_output` must be burned.

Any other burned tokens are irrelevant, and the `burn_everyting` spending validator ensures that no tokens are minted.

3.8.2 Spending validator

The `burn_everything` validator requires every non-ADA token in the transaction to be burned. Conditions:

1. The transaction must have a single output.
2. The transaction's output must only contain ADA.

Chapter 4

Proof protocol

4.1 Fraud proof catalogue

Midgard uses a key-unordered linked list (see Appendix A.1.9) to track all possible categories of fraud that can occur in Midgard blocks. Every enforced Midgard ledger rule (see Chapter 5) must be represented by a fraud proof category indicating the violation of that rule.

Each node in the `fraud_proof_catalogue` specifies the script hash of the first step in a fraud proof computation (see Section 4.3), which inspects a given block and succeeds if the block has an instance of the fraud category. The node keys are sequentially ordered and 4 bytes each, allowing the catalogue to track 4096 fraud proof categories.

$$\text{FraudProofCatalogueDatum} := \left\{ \text{init\_step} : \quad \text{ScriptHash} \right\}$$

4.1.1 Minting policy

The `fraud_proof_catalogue` minting policy is statically parametrized on the `hub_oracle` minting policy and the Midgard governance key.[1] Redeemers:

Init. Initialize the `fraud_proof_catalogue` via the Midgard hub oracle. Conditions:

1. The transaction must mint the Midgard hub oracle NFT.
2. The transaction must Init the `fraud_proof_catalogue` list.

Deinit. Deinitialize the `fraud_proof_catalogue` via the Midgard hub oracle. Conditions:

1. The transaction must burn the Midgard hub oracle NFT.
2. The transaction must Deinit the `fraud_proof_catalogue` list.

New Fraud Category. Catalogue a new Midgard fraud category. Conditions:

[1] For now, Midgard governance is centralized on a pubkey. This should be replaced by the proper governance mechanism when it is specified.

1. The transaction must Append a node to the `fraud_proof_catalogue`. Let that node be `new_node`.

2. The Midgard governance key must sign the transaction.

3. Let `old_node` be the node that links to `new_node` in the transaction outputs.

4. The `new_node` key must be four bytes long.

5. If `old_node` is the root node:

   - The `new_node` key must be zero.

6. Otherwise:

   - The `old_node` key must be less than 4095.

   - The `new_node` must be larger than `old_node` key by one.

Remove Fraud Category. Remove a Midgard fraud category. Conditions:

1. The transaction must Remove a node from the `fraud_proof_catalogue`.

2. The Midgard governance key must sign the transaction.

4.1.2 Spending validator

The spending validator of `fraud_proof_catalogue_addr` is statically parametrized on the `fraud_proof_catalogue` minting policy. Conditions:

1. The transaction must burn a `fraud_proof_catalogue` token.

2. The transaction must *not* mint or burn any other tokens.

4.2 Fraud proof tokens

Fraud proof tokens represent fraud proof computations that have successfully concluded.

4.2.1 Minting policy

The `fraud_proof` minting policy is statically parametrized on the `computation_thread` and `hub_oracle` minting policies. Redeemers:

Mint. Mint a new fraud proof token whenever a fraud proof computation succeeds. Conditions:

1. The transaction must burn a `computation_thread` token.
2. The transaction must mint a `fraud_proof` token with the same token name as the `computation_thread` token.
3. The transaction must include the Midgard hub oracle NFT in a reference input.
4. Let `fraud_proof_addr` be the policy ID in the corresponding field of the Midgard hub oracle.
5. The `fraud_proof` token must be sent to the `fraud_proof_addr` spending validator.

4.2.2 Spending validator

The spending validator of `fraud_proof_addr` does *not* allow its utxo to be spent. Midgard fraud proofs last forever.

4.3 Fraud proof computation threads

A computation thread splits up a large computation into a series of steps, passing control between the steps in the continuation-passing style (CPS). In other words, the computation thread is a state machine (see Appendix A.2) with a linear state graph.

Each step is a spending validator that executes the following sequence:

1. Parses the computation state from its datum.
2. Optionally receives arguments from its redeemer to guide the computation step.
3. Advances the computation by the step, producing a new computation state.
4. Suspends the computation by serializing the new computation state into a new datum that it sends to the spending validator of the next step.

All steps in a computation thread parametrize the same datum type:

$$\mathsf{StepDatum}(\mathsf{state\_data}) := \left\{ \begin{array}{ll} \mathsf{fraud\_prover} : & \mathsf{PubKeyHash} \\ \mathsf{data} : & \mathsf{state\_data} \end{array} \right\}$$

4.3.1 Minting policy

The `computation_thread` minting policy initializes its state machine. It is statically parametrized on the `fraud_proof_catalogue` and `hub_oracle` minting policies. Redeemers:

Init. Conditions:

1. The transaction must reference a node of the `fraud_proof_catalogue`. Let that node be `fraud_category`.
2. The transaction must include the Midgard hub oracle NFT in a reference input.
3. Let `state_queue` be the policy ID in the corresponding field of the Midgard hub oracle.
4. The transaction must reference a node from the `state_queue`. Let `fraud_node` be that node.
5. The transaction must mint a single token of the `computation_thread` minting policy. The token name must concatenate the four-byte key of `fraud_category` and the 28-byte block header hash in the key of `fraud_node`.
6. The computation thread token must be sent to the spending validator defined in the `init_step` field of `fraud_category`. Let `output_state` be that transaction output.
7. The `output_state` datum type must be `StepDatum(Void)`.[2]
8. The transaction must be signed by the `fraud_prover` pub-key hash of `output_state`.
9. Other than ADA, `output_state` must *not* hold any other tokens.
10. The transaction must *not* mint or burn any other tokens.

[2] Aiken calls it StepDatum(Void), while Plutarch calls it StepDatum(PUnit).

Success. Terminate the state machine normally from the final spending validator in the computation. There are no conditions because this redeemer relies on the spending validator to burn the token.

Cancel. Terminate the state machine exceptionally from any spending validator in the computation. There are no conditions because this redeemer relies on the spending validator to burn the token.

4.3.2 Spending validators

A fraud-proof computation succeeds if its thread token passes through all the steps' spending validator addresses. In that case, the last step's spending validator reifies the successful fraud-proof by requiring a fraud-proof token to be minted. The fraud-proof minting policy requires the computation thread token to be burned, specifically via the Success redeemer.

On the other hand, at any step, the person who initiated the computation thread can cancel the computation instead of advancing it. In that case, the step's spending validator requires the computation thread token to be burned via the Cancel redeemer. Thus, while there is typically only one path for a computation thread to reach success via the sequential steps,[3] there may be multiple opportunities for the computation to be canceled along the way.

For each fraud-proof category, each spending validator is custom-written to express the specific logic of that computation step, and it is statically parametrized on the next step's spending validator (if any). One of the custom conditions of the computation step should verify the transition between the input state and output state of the thread:

$$\begin{aligned} \mathsf{verify\_transition} &: (\mathsf{Input}, \mathsf{Output}, ..\mathsf{Args}) \rightarrow \mathsf{Bool} \\ \mathsf{verify\_transition}(\mathsf{i}, \mathsf{o}, ..\mathsf{args}) &:= \Big(\mathsf{transition}(\mathsf{i}, ..\mathsf{args}) \equiv \mathsf{o}\Big) \\ \mathsf{transition} &: (\mathsf{Input}, ..\mathsf{Args}) \rightarrow \mathsf{Output} \end{aligned}$$

All of the spending validators share the same parametric redeemer type, but each spending validator can parametrize the Continue redeemer by a different type to hold the custom instructions needed to guide the computation step:

$$\mathsf{StepRedeemer}(\mathsf{instructions}) := \mathsf{Continue}(\mathsf{instructions}) \mid \mathsf{Cancel}$$

These redeemers should be handled in the following general pattern:

Continue. Advance the computation. Conditions:

1. If this is the last step of the computation:
    - Mint the fraud token, which will implicitly burn the computation thread token with the Success redeemer. Let `output_state` be that transaction output.
    - The `output_state` datum type must be `StepDatum(Void)`.

[3] Technically, if there are multiple instances of the same fraud category in a fraudulent block, then there is a corresponding number of paths to prove the occurrence of that fraud category in the block. The redeemer arguments provided to the computation steps collectively select one of these paths.

   - The `fraud_prover` field must match between the `output_state` and the input datum.

2. Otherwise:

   - The computation thread token must be sent to the next step's spending validator. Let `output_state` be that transaction output.

   - The `fraud_prover` field must match between the `output_state` and the input datum.

3. Evaluate the custom conditions of the computation step, including verifying the state transition.

4. The custom conditions may require the transaction to reference a state queue with a key hash matching the last 28 bytes of the computation thread token name.

5. The transaction must *not* mint or burn any other tokens.

Cancel. Cancel the computation. Conditions:

1. Burn the computation thread token with the Cancel redeemer.

2. Return the ADA from the computation thread utxo to the fraud prover pub-key defined in the input datum.

3. The transaction must *not* mint or burn any other tokens.

Chapter 5

Ledger rules and fraud proofs

5.1 Midgard Ledger Rules and Fraud Proofs

In the following sections the following premises are used:

$$
\begin{aligned}
b &\in Blocks \\
txs &:= transactions(b) \\
utxos_{pre} &:= prev\_utxos(b) \\
utxos_{post} &:= utxos(b) \\
wtxs &:= withdrawals(b)
\end{aligned}
$$

5.1.1 Rule: All inputs must be valid

A transaction cannot spend a non-existing (or an already spent) UTxO. Formal specification:

$$
\begin{aligned}
\forall t \in Ledger, \forall i \in spend\_inputs(t) :& \\
&(\exists t_1 \in Ledger, t \neq t_1 \land i \in outputs(t_1)) \land \\
&(\nexists t_2 \in Ledger, t \neq t_2 \land i \in spend\_inputs(t_2))
\end{aligned}
$$

This ledger rule is violated if any of the following violations occur:

- NO-INPUT
- INPUT-NO-IDX
- WITHDRAWN-INPUT
- DOUBLE-SPEND
- DOUBLE-WITHDRAW

NO-INPUT violation

A transaction t attempted to spend the UTxO i that does not exist or was spent in a previous block. Formal specification:

$$
\begin{aligned}
\exists t \in txs, \exists i \in spend\_inputs(t) :& \\
&(i \notin utxos_{prev}) \land \\
&(\nexists t_1 \in txs, t \neq t_1 \land tx\_hash(t_1) = tx\_hash(i))
\end{aligned}
$$

Fraud proof construction:

1. Let t be the transaction alleged to violate the ledger rule.
2. A membership proof must be provided for t that shows that the transaction is included in the block ($t \in txs$).
3. A membership proof for input i must be specified such that $i \in spend\_inputs(t)$.
4. A non-membership proof must be created to show that i is not in $utxos_{prev}$.
5. A non-membership proof must also be generated that shows that there are no transactions in txs that have id $tx\_hash(i)$.

INPUT-NO-IDX violation

A transaction t attempted to spend the input i, which did not exist at all. Formal specification:

$$\exists t \in txs, \exists i \in spend\_inputs(t) : \\ (\exists t_1 \in txs, tx\_hash(t_1) = tx\_hash(i) \wedge i \notin outputs(t_1))$$

Fraud proof construction:

1. Let t be the transaction alleged to violate the ledger rule.
2. A membership proof must be provided for t that shows that the transaction is included in the block ($t \in txs$)
3. A membership proof must be presented for input i such that $i \in spend\_inputs(t)$
4. A membership proof must be created for t_1 such that $tx\_hash(t_1) = tx\_hash(i)$
5. A DA layer proof must be presented that certifies that $length(outputs(t_1)) < index(i)$ TODO

WITHDRAWN-INPUT violation

A transaction t attempted to spend the input i, which was spent in a withdraw transaction. Formal specification:

$$\exists t \in txs, \exists i \in spend\_inputs(t), \exists w \in wtxs, i = l2\_outref(wtx)$$

Fraud proof construction:

1. Let t be the transaction alleged to violate the ledger rule.
2. A membership proof must be provided for t that shows that the transaction is included in the block ($t \in txs$)
3. A membership proof must be generated for input i such that $i \in spend\_inputs(t)$
4. A membership proof must be created to show that w is in $wtxs$, which also spends input i

DOUBLE-SPEND violation

A transaction t attempted to spend the input i, which was spent in another transaction. Formal specification:

$$\exists t \in txs, \exists i \in spend\_inputs(t), \exists t_1 \in tx, t \neq t_1 \wedge i \in spend\_inputs(t_1)$$

Fraud proof construction:

1. Let t be the transaction alleged to violate the ledger rule.
2. A membership proof must be provided for t that shows that the transaction is included in the block ($t \in txs$)
3. A membership proof must be generated for input i such that $i \in spend\_inputs(t)$
4. A membership proof must be created to show that t_1 is in txs
5. A membership proof must be given that verifies that $i \in spend\_inputs(t_1)$

DOUBLE-WITHDRAW violation

A withdrawal w attempted to spend the same input, which was already withdrawn by w_1. Formal specification:

$$\exists w, w_1 \in wtxs, w \neq w_1 \wedge l2\_outref(w) = l2\_outref(w_1)$$

Fraud proof construction:

1. Let t be the transaction alleged to violate the ledger rule.
2. A membership proof must be provided for w that shows that the withdraw transaction is included in the block ($w \in wtxs$)
3. A membership proof must be created to show that w_1 is in $wtxs$

5.1.2 Rule: Transaction validity range

Every valid transaction in the ledger must be included at a timestamp that conforms to the validity range that the transaction prescribes. Formal specification: **TODO**

$$\forall t \in Ledger, time\_range(block(t)) \subseteq validity\_interval(t)$$

This ledger rule is violated if any of the following violations occur:

- INVALID-RANGE

INVALID-RANGE violation

A transaction t has a time-validity range that does not overlap with its block's event interval. Formal specification: TODO

$$\exists t \in txs, time\_range(b) \not\subseteq validity\_interval(t)$$

Fraud proof construction:

1. Let t be the transaction alleged to violate the ledger rule.
2. A membership proof must be provided that proves that transaction t is included in the block b

5.1.3 Rule: At least one input

Every valid transaction in the ledger must spend at least one UTxO. Formal specification:

$$\forall t \in Ledger, |spend\_inputs(t)| > 0$$

This ledger rule is violated if any of the following violations occur:

- ZERO-INPUT

ZERO-INPUT violation

A transaction t is in the ledger, while $|inputs(t)| = 0$ Formal specification:

$$\exists t \in txs, |spend\_inputs(t)| = 0$$

Fraud proof construction:

1. Let t be the transaction alleged to violate the ledger rule.
2. A membership proof must be provided that shows that the transaction is included in the ledger
3. A DA layer proof must be presented that certifies that $length(spend\_inputs(t)) = 0$ TODO

5.1.4 Rule: Minimum fee

Every valid transaction in the ledger must pay the fees for inclusion. Formal specification:

$$\forall t \in Ledger, tx\_fee(t) \geq min\_fee(t)$$

The fee calculation algorithm is the same as in Cardano ... TODO

This ledger rule is violated if any of the following violations occur:

- MIN-FEE

MIN-FEE violation

A transaction t is in the ledger, while $fee(t) < min\_fee(t)$. Formal specification:

$$\exists t \in txs, fee(t) < min\_fee(t)$$

Fraud proof construction:

1. Let t be the transaction alleged to violate the ledger rule.
2. A membership proof must be provided that shows that the transaction is included in the ledger

5.1.5 Rule: Required signatures are correct

Every valid transaction in the ledger must correctly show the required signers. Formal specification:

$$\begin{aligned}&\forall t \in Ledger, required\_signer\_hashes(t) = \\ &\quad \{\, paymentHK(addr(u)) \mid (r, u) \in utxos_{post}, r \in spend\_inputs(t), addr(u) \in Addr^{vkey} \,\}\end{aligned}$$

This ledger rule is violated if any of the following violations occur:

- MISSING-REQ-SIGNER
- NON-REQ-SIGNER

MISSING-REQ-SIGNER violation

A transaction t is in the ledger and it violates the "Required signatures" property. Formal specification:

$$\begin{aligned}&\exists t \in txs, \exists r \in spend\_inputs(t), \exists u \in Output, \\ &\quad (r, u) \in utxos_{post} \wedge paymentHK(addr(u)) \notin required\_signer\_hashes(t) \wedge \\ &\quad addr(u) \in Addr^{vkey}\end{aligned}$$

Fraud proof construction:

1. Let t be the transaction alleged to violate the ledger rule.
2. A membership proof must be provided that shows that the transaction is included in the ledger
3. A DA layer proof must be presented that shows that $r \in spend\_inputs(t)$
4. A membership proof must be generated that shows that $(r, u) \in utxos_{post}$
5. A DA layer proof must be shown that proves that $paymentHK(addr(u)) \notin required\_signer\_hashes(t)$

NON-REQ-SIGNER violation

A transaction t is in the ledger and it violates the "Required signatures" property. Formal specification:

$$\begin{aligned}&\exists t \in txs, \exists vkey \in required\_signer\_hashes(t), \nexists (r, u) \in utxos_{post},\\ &\quad r \in spend\_inputs(t) \land paymentHK(addr(u)) = vkey \land addr(u) \in Addr^{vkey}\end{aligned}$$

Fraud proof construction:

1. Let t be the transaction alleged to violate the ledger rule.
2. A membership proof must be provided that shows that the transaction t is included in the ledger
3. A DA layer proof must be presented that shows that for a specified $vkey$ there is no utxo $u \in utxos_{post}$, such that the address of u corresponds to $vkey$ and t spends u

5.1.6 Rule: Signatures are valid

Every provided signature must be valid. Formal specification:

$$\forall t \in Ledger, \forall (v, s, h) \in addr\_tx\_wits(t), is\_valid\_signature(v, s, h)$$

This ledger rule is violated if any of the following violations occur:

- INVALID-SIGNATURE

INVALID-SIGNATURE violation

There exists an invalid signature for transaction t (if indeed the signature (v, s, h) is not valid). Formal specification:

$$\exists t \in txs, \exists (v, s, h) \in addr\_tx\_wits(t), \neg is\_valid\_signature(v, s, h)$$

Fraud proof construction:

1. Let t be the transaction alleged to violate the ledger rule.
2. A membership proof must be provided that shows that the transaction t is included in the ledger
3. A membership proof must be shown that states that $(v, s, h) \in addr\_tx\_wits(t)$

5.1.7 Rule: Every needed signature is provided

Every required signature is provided. Formal specification:

$$\forall t \in Ledger, \forall h \in required\_signer\_hashes(t), \exists (v, s, h) \in addr\_tx\_wits(t)$$

This ledger rule is violated if any of the following violations occur:

- MISSING-SIGNATURE

MISSING-SIGNATURE violation

A required signature, corresponding to h is missing. Formal specification:

$$\exists t \in txs, \exists h \in required\_signer\_hashes(t), h \notin \{h_p \mid (v, s, h_p) \in addr\_tx\_wits(t)\}$$

Fraud proof construction:

1. Let t be the transaction alleged to violate the ledger rule.
2. A membership proof must be provided that shows that the transaction t is included in the ledger
3. A DA layer proof must be shown that states that $h \in required\_signer\_hashes(t)$
4. A DA layer proof must be presented that shows that a signature with h does not exist

5.1.8 Rule: Multisig scripts are available

5.1.9 Rule: Multisig scripts validated

5.1.10 Rule: Value preservation

The total value must be preserved. Formal specification:

$$\begin{aligned}&\forall t \in Ledger,\\ &mint(t) + \sum_{i \in spend\_inputs(t)} value(i) = fee(t) + \sum_{o \in outputs(t)} value(o)\end{aligned}$$

5.1.11 Rule: No Ada minted

Ada must not be minted. Formal specification:

$$\forall t \in Ledger, lovelaces(mint(t)) = 0$$

5.1.12 Rule: Maximum transaction size

5.1.13 Rule: Maximum reference script size

5.1.14 Rule: Maximum value size

5.1.15 Rule: No negative value

All values must be greater or equal to zero. Formal specification:

$$\forall t \in Ledger, mint(t) \geq \mathbf{0} \wedge fee(t) \geq 0 \wedge \forall o \in outputs(t),\ value(o) \geq \mathbf{0}$$

5.1.16 Rule: Minimum UTxO value

5.1.17 Rule: Network id of inputs

5.1.18 Rule: Network id of reference inputs

5.1.19 Rule: Network id of outputs

5.1.20 Rule: Network id of withdrawals

5.1.21 Rule: Network id of transaction

5.1.22 Rule: All reference inputs must be valid

5.1.23 Rule: Validator scripts are available

5.1.24 Rule: Validator scripts adhere to execution limits

5.1.25 Rule: Validator scripts accept transaction

5.1.26 Rule: Maximum number of collateral inputs

5.2 Custom Midgard ledger rules

5.2.1 Rule: Is valid

5.2.2 Rule: No auxiliary data

5.2.3 Rule: Certificates empty

5.2.4 Rule: Only zero withdrawals

5.2.5 Rule: No mints

5.3 Data availability rules

5.3.1 Rule: Transaction count correct

Chapter 6

Offchain data architecture

Midgard operators only commit block headers to the L1 state queue. They store their actual blocks temporarily in Midgard's data availability layer for at least the maturity period, and then permanently on Midgard's archive nodes.

6.1 Data availability layer

The data availability layer is critical to Midgard's security because every committed block needs to be publicly available throughout the maturity period so that watchers can detect and prove fraud before invalid blocks are merged.

We are considering three solutions for the data availability layer (in decreasing preference):

1. Cardano Leios blobs (the ideal solution)
2. Multi-signature committees

6.1.1 Data availability via Leios blobs

The ideal data availability solution for Midgard is based on Cardano Leios blobs, which are a proposed feature in Cardano that will support large-scale transient data storage secured via L1 consensus.

The contents of these blobs do not have to be verified by Cardano's Ouroboros consensus protocol, and they only have to be stored for up to 30 days. This means that a large amount of data can be stored in these blobs sustainably for a low cost, which we expect to be several orders of magnitude lower than Cardano's cost for transaction metadata (which is stored permanently).

Leios blobs are a natural intermediate point on Leios' multi-year roadmap toward full input-endorser capabilities. We believe that they are achievable before Midgard's planned deployment on mainnet, and we will support the Leios team to help bring them to Cardano sooner.

In the Leios-based data availability solution for Midgard, operators will pay to store their full non-Merkelized blocks inside Leios blobs for the full maturity period. Leios itself will provide timestamps and (non-Merkle) hashes for the blobs and ensure that the blob contents are accessible. Midgard's L1 smart contracts will be able to access the timestamps and non-Merkle hashes of blobs directly.

The block data inside each Leios blob will be sufficient to be converted offchain into the Merkelized representation of Midgard blocks that is necessary to construct fraud proofs. The correspondence between the non-Merkelized block data stored in the Leios blob and the Merkle root hash declared in the block header will be verifiable via a special fraud proof verification procedure. This procedure will calculate the Merkle root hash in a streaming fashion over the block data and compare it to the declared Merkle root hash in the block header.

> We hope to further streamline this Merkle root hash verification procedure by collaborating with the Cardano Leios and Plutus teams to make it a Plutus builtin operation. This builtin can be much more expensive than typical Plutus built-ins, and Midgard fraud provers will be happy to pay the cost because the stakes of DA fraud and the reward for proving it are much higher. Nonetheless, this will not impose an ongoing cost burden on Midgard because this verification procedure will not need to be invoked during normal operation with honest blocks, and invalid blocks will be rare.

Operators' costs for storing blocks in Leios will be offset by the revenue they collect from Midgard transaction, deposit, and withdrawal fees. Furthermore, the Leios blob storage fees will become an additional source of revenue for Cardano L1 block producing nodes, further boosting the economic security of Cardano L1 on which Midgard depends.

6.1.2 Data availability via multi-signature committee

Our fallback solution, in case Leios blobs are infeasible or take too long to reach Cardano mainnet, is to use stake-weighted multi-signature committees. This will operate in a similar way to the Celestia data availability layer and similar protocols.

Operators will store block data in their declared publicly accessible locations. Watchers of the data availability layer will perform regular Data Availability Sampling (DAS) to check that the data is indeed available at those locations. This sampling will be conducted on a regular basis by all light-client nodes that access Midgard L2 data (for users), and also by dedicated watchers that monitor Midgard for the chance to earn fraud prover rewards.

If multiple watchers detect data unavailability during a block's maturity period, they will generate a stake-weighted multi-signature to attest that the data is unavailable. If the stake-weight of the multi-signed attestation exceeds Midgard's `da_multisig_threshold` parameter, it can be posted on Cardano L1 to disqualify the operator's block and slash the operator's bond.

This threshold parameter will need to be calibrated so that it is small enough to allow operators to be penalized for data unavailability and large enough that a small group of malicious watchers cannot frivolously penalize honest Midgard operators.

6.2 Archive node

Midgard's archive nodes will store the block data for confirmed blocks. Security properties are easier to achieve for this historical data because, at all times, Midgard's confirmed state utxo contains a chained header hash that pins the entire historical chain of confirmed blocks. There can be no disagreement about different versions of this data—an archive node either stores data that corresponds to the confirmed state header hash, or it does not.

We expect all operators to keep a copy of the latest confirmed block's utxo set because it is necessary for them to create honest blocks and avoid being slashed for fraudulent blocks. They are incentivized to do so by the fee revenue they earn on Midgard. This utxo set and the block data stored in the data availability layer provide Midgard with the minimum information necessary to continue producing blocks.

We expect indexers and other professional service providers to store copies of the Midgard historical data. Midgard's security and ongoing block-production capability does not depend on this historical data. However, we expect it to be stored to the extent that users are willing to pay for this service.

Branch: gf/spec-security @ c945928 • Commit date: 2025-01-18

Chapter 7

Phase Two Validation

Phase two validation represents the critical stage in Midgard's transaction processing where UPLC script evaluation occurs.

7.1 Overview

Phase two validation serves several key purposes:

- Validates the execution of UPLC scripts attached to transactions
- Ensures computational bounds are respected
- Maintains verifiable execution traces for fraud proof construction
- Enables efficient dispute resolution through progressive state hashing

The validation process consists of three main components:

1. State representation and management
2. Off-chain script decoding and preparation
3. Execution validation and fraud proof mechanisms

Each component is designed to maintain verifiability while optimizing for L2 performance constraints. The following sections detail these components and their interactions within the Midgard protocol.

7.1.1 Validation Requirements

For a transaction to pass phase two validation:

- All scripts must be successfully decoded from their byte representation
- Script execution must complete within specified resource bounds
- All state transitions must be cryptographically verifiable
- Execution traces must enable efficient fraud proof construction

7.2 State Representation

This section describes how the state is represented during phase two validation, including the data structures and encoding methods used for UPLC evaluation.

7.2.1 Term Representation

The UPLC Term representation uses progressive hashing to maintain compact state while preserving verifiability:

$$\begin{aligned}\text{Term} := {} & \text{Variable}(\text{Name}) \\ & \mid \text{Lambda}(\text{Name}, \text{Hash}) \\ & \mid \text{Apply}(\text{Hash}, \text{Hash}) \\ & \mid \text{Constant}(\text{Hash}) \\ & \mid \text{Force}(\text{Hash}) \\ & \mid \text{Delay}(\text{Hash}) \\ & \mid \text{Builtin}(\text{BuiltinFunction})\end{aligned}$$

Each Hash in the Term representation refers to another Term that has been previously processed and hashed. This creates a directed acyclic graph (DAG) of Term components where larger structures are decomposed into their constituent parts.

For example, a lambda expression like $\lambda x.\lambda y.[yx]$ would be represented as:

- A Lambda node containing (x, h_1)
- Where h_1 is the hash of a Lambda node containing (y, h_2)
- Where h_2 is the hash of an Apply node containing (h_3, h_4)
- Where h_3 is the hash of a Variable node containing y
- Where h_4 is the hash of a Variable node containing x

7.2.2 Decoding Steps

The conversion from flat-encoded script bytes to the final Term follows a sequence of BytesToTermSteps:

$$\text{BytesToTermStep} := \left\{ \begin{array}{ll} \text{remaining\_bytes}: & \text{ScriptBytes} \\ \text{partial\_term}: & \text{Term} \end{array} \right\}$$

Each step represents an atomic transformation in the decoding process, with the decoding_step_proof enabling independent verification of that specific step.

7.2.3 Execution Steps

The execution state during UPLC evaluation is represented by the CEK machine state:

$$\text{CEKState} := \left\{ \begin{array}{ll} \text{term\_hash}: & \text{Hash} \\ \text{env}: & \text{Environment} \\ \text{continuation}: & \text{Continuation} \end{array} \right\}$$

Each execution step produces a new CEK state and tracks execution units consumed:

$$\text{ExecutionStep} := \left\{ \begin{array}{ll} \text{before\_state}: & \text{CEKState} \\ \text{after\_state}: & \text{CEKState} \\ \text{execution\_units}: & \text{ExecutionUnits} \end{array} \right\}$$

7.2.4 Execution Trace

The complete execution trace combines the bytes-to-term conversion and CEK machine evaluation. In the following, we use the notation $\mathcal{RH}$ to indicate a root hash of a Merkle-Patricia tree.

$$\text{ExecutionTrace} := \left\{ \begin{array}{ll} \text{bytes\_to\_term\_steps}: & \mathcal{RH}(\text{[BytesToTermStep]}) \\ \text{initial\_state}: & \text{CEKState} \\ \text{steps}: & \mathcal{RH}(\text{[ExecutionStep]}) \end{array} \right\}$$

The execution trace is stored in transaction witness sets:

$$\text{MidgardTxWits} := \left\{ \begin{array}{ll} \ldots & \\ \text{execution\_traces}: & ?\ \mathcal{RH}(\text{Map(RdmrPtr, ExecutionTrace)}) \end{array} \right\}$$

This representation allows any step of the trace to be independently verified without requiring access to the complete execution history, enabling efficient fraud proof construction and validation.

7.3 Off-chain Decoding

This section details the process of decoding UPLC script bytes into executable Terms during phase two validation.

7.3.1 Byte Format Specification

The exact byte format specification for UPLC scripts will be detailed in a future update, following thorough verification of the encoding scheme.

7.3.2 Decoding Process

7.3.3 Term Construction Process

7.3.4 Transformation Types

The specific byte format and transformation rules for different Term types will be detailed in a future specification update, after thorough verification of the encoding scheme.

7.4 Fraud Proofs in UPLC Evaluation

This section details the fraud proofs involved in UPLC evaluation, focusing on single-step verification of state transitions.

7.4.1 Types of Fraud Proofs

The system supports these categories of fraud proofs:

Decoding Fraud Proofs of invalid script byte decoding:

- Invalid byte format
- Size limit violations
- Reference resolution failures

Execution Fraud Proofs of invalid UPLC execution:

- Resource limit violations
- Invalid state transitions
- Incorrect execution results

7.4.2 Single-Step Verification

The fraud proof system verifies individual state transitions:

$$\forall s_1, s_2 \in \text{CEKState} : \text{claimed\_transition}(s_1 \to s_2) \text{ valid} \iff \text{compute\_next\_state}(s_1) = s_2$$

To prove a violation:

- The operator provides the claimed before state (s_1) and after state (s_2)
- The challenger computes the actual next state from s_1
- If the computed state differs from s_2, the transition is invalid
- No additional trace information is required

7.4.3 Proof Data Structure

The fraud proof structure is minimal:

$$\text{FraudProof} := \left\{ \begin{array}{ll} \text{step\_number} : & \mathbb{N} \\ \text{before\_state} : & \text{CEKState} \\ \text{claimed\_after\_state} : & \text{CEKState} \\ \text{actual\_after\_state} : & \text{CEKState} \end{array} \right\}$$

7.4.4 Verification Process

The verification process is straightforward:

1. Verify the before state matches the operator's claim
2. Compute one step from the before state
3. Compare computed result with operator's claimed after state
4. If they differ, the fraud proof is valid

7.4.5 Security Considerations

The single-step verification approach provides several benefits:

- Minimal proof size
- Constant-time verification
- No complex challenge periods needed
- Deterministic outcomes

Security guarantees include:

- No false positives (valid transitions cannot be proven invalid)
- No trace reconstruction required
- Immediate verification of claims
- Protection against computational waste attacks

Appendix A

General onchain data structures and mechanisms

This appendix describes general-purpose data structures and mechanisms that can be used for various applications on Cardano. They are designed to be as simple and flexible as possible, and they do not depend on any specifics of the Midgard protocol.

A.1 Linked list

The linked list is a versatile data structure that can implement sets, queues, key-value maps, lazy data sequences, and other interesting data structures. It is particularly useful in the extended UTXO model of the Cardano blockchain, where many list operations and queries can be validated onchain within minimal local contexts that do not grow with list size. Meanwhile, offchain queries can access any part of the list via beacon NFTs that pinpoint specific list nodes.

We describe two types of linked lists:

- The **key-unordered** linked list is a cheaper/simpler variant that only allows nodes to be appended at the end of the list and does not enforce the order of keys in the list. It can be used to implement queues and other sequential data structures. However, appending to a list is a sequential operation, which is unsuitable for applications that need multiple independent actors to grow the list simultaneously.
- The **key-ordered** linked list allows nodes to be inserted anywhere in the list, but it applies additional rules to maintain the key-ascending order of nodes. It can be used to implement sets and key-value maps. List insertion is increasingly parallel as the list grows, so the key-ordered linked list is well-suited to applications with multiple independent actors growing the list (see Appendix A.1.12).

A.1.1 Utxo representation

Each linked list, whether key-ordered or key-unordered, is represented in the blockchain ledger as a collection of node utxos that hold node NFTs minted by the list's minting policy and are held under the list's spending validator. The spending validator is parameterized by the minting policy, and the minting policy is parameterized by the utxo reference of one of the spent inputs in the transaction that initializes the list.

Each node utxo of a list uniquely corresponds to some `ByteArray` key and holds a node NFT with a token name equal to that key's serialization. The key-ordered list's state transitions guarantee that all nodes have unique keys, but the key-unordered list's state transitions only assume that keys are unique.

> An application using a key-unordered list MUST only add nodes with unique keys to the list. Duplicate keys break the linked list data structure.

Every node utxo has a datum of the `NodeDatum` type, which is parametric on the `app_data` type of the non-key data that is to be stored in the list nodes:

$$\mathsf{NodeDatum}(\mathsf{app\_data}) := \left\{ \begin{array}{ll} \mathsf{key}: & \mathsf{Option}(\mathsf{ByteArray}) \\ \mathsf{link}: & \mathsf{Option}(\mathsf{ByteArray}) \\ \mathsf{data}: & \mathsf{app\_data} \end{array} \right\}$$

The `key` field indicates the node's key (or `None` for the root node). [1] It must match the token name of the node's NFT when serialized:[2]

$$\begin{aligned} \mathsf{serialize\_key}(\mathsf{None}) &:= \text{“Node''} \\ \mathsf{serialize\_key}(\mathsf{Some}(\mathsf{key})) &:= \mathsf{concat}(\text{“Node''}, \mathsf{key}) \end{aligned}$$

The root node of a list is the only node with a `key` field value of `None` in its `NodeDatum` and its NFT token name set to `‘‘Node’’`. The root node is created when a list is initialized and must continue to exist until the list is deinitialized. This allows an initialized but empty list to be represented by just its root node (and no other nodes). The non-root nodes represent the actual keys and data of interest within a non-empty list.

The `link` field is a link to the next key that a forward traversal through the list must visit after visiting the current node. Alternatively, the `link` field can be interpreted as the previous key that a backward traversal through the list has visited before the current node. The last node is the only node with a `link` field value of `None` because forward traversal through the list must end after this node (backward traversal must start at this node).

The `data` field is parametric on the `app_data` type set by the application that instantiates the list. It is reserved for use by the application's custom onchain code.

A.1.2 Linked list library structure

The linked list library provides a collection of *rules* that enforce the baseline behavior of linked lists. An application using the list data structure should invoke one of these rules whenever it needs a particular linked list operation to occur within the context of the application's custom state transitions.

---

[1] The key field is redundant, as it is already represented in the node NFT, but we include it in the datum for convenience in onchain scripts and offchain analytics.

[2] Prefixing the key when it is serialized to the token name prevents a collision between the root node and a node with an empty key string. It also allows more flexibility for minting policies that use the linked list library to atomically mint additional tokens associated with the key but namespaced from the node NFTs. Alternatively, if this functionality is not needed, the “Node” prefixes can be removed from the serialization.

Linked list rules are predicate functions that can be freely composed with any custom application rules via logical conjunction. In this way, while the linked list rules control which operations can be performed on a list and how they must be performed, applications can apply further constraints on the circumstances under which the operations are allowed.

The state transition rules control how lists are initialized/deinitialized and how nodes are added/removed from the list. These rules should be invoked in the application's minting policy for the list because every state transition of the list changes the number of list nodes, so it must necessarily mint or burn node NFTs.

The state integrity rules ensure that node NFTs stay in their respective node utxos until burned and that the `key` and `link` fields remain unchanged outside of linked list state transitions. These rules should be invoked in the application's spending validator for the list.

Neither the state transition nor the state integrity rules inspect the `data` field of node utxos, nor do they inspect the value held by node utxos besides node NFTs. Applications are free to use these as they wish.

Appendix A.1.10 describes how to instantiate a linked list within an application and compose the linked list rules with custom app rules.

A.1.3 State integrity rules

The state integrity rules distinguish between transactions that apply linked list state transitions and transactions that merely modify the `data` field or non-node-NFT value of a node utxo.

List State Transition. This rule allows a node utxo to be spent in any transaction that applies a state transition to the list. Conditions:

1. Tokens of the list's minting policy must be minted or burned.

Modify Data. This rule ensures that the node NFT, the `key` field, and the `link` field remain unchanged in any transaction that does not apply a state transition to the list. It achieves it as follows:

1. Tokens of the list's minting policy must *not* be minted or burned.
2. Let `own_input` be the node utxo input for which this rule is being evaluated.
3. Let `own_output` be an output of the transaction indicated by a redeemer argument of the spending validator.
4. The node NFT of the list must be present in `own_input`.
5. The value must match in `own_input` and `own_output`.
6. The `key` and `link` fields must match in `own_input` and `own_output`.

To keep the onchain code simple and efficient, this rule can only handle content modifications to one node utxo per list. In principle, this could be generalized to handle bulk modifications of node contents, but that would require more expensive computations to match node inputs with node outputs.

A.1.4 List subcontext for state transition rules

State transition rules of a linked list are only concerned with a transaction's effects on the list's node utxos. Consequently, they focus on the following linked list subcontext, which is a subset of the overall transaction context:

- `node_cs` is the minting policy ID of the list.
- `node_mint` is the map of tokens of the list's minting policy that were minted or burned.
- `node_inputs` are the transaction inputs that hold node NFTs of the list and datums that parse as `NodeDatum`.
- `node_outputs` are the transaction outputs that hold node NFTs of the list and datums that parse as `NodeDatum`.

The state transition rules ignore the rest of the transaction context, which does not affect the state of the list. However, applications invoking the state transition rules are free to consider the whole transaction context to constrain the circumstances under which they allow linked list state transitions to occur.

A.1.5 Node and key predicates

The following predicate functions characterize nodes and keys in a linked list subcontext relative to the global state of the list.

Root node. The root node of the list:

$$\mathsf{is\_root\_node}(\mathsf{node}) := (\mathsf{node.datum.key} \equiv \mathsf{None})$$

Last node. The last node of the list:

$$\mathsf{is\_last\_node}(\mathsf{node}) := (\mathsf{node.datum.link} \equiv \mathsf{None})$$

Empty list. The list is empty if its root node is also its last node:

$$\mathsf{is\_root\_node}(\mathsf{node}) \land \mathsf{is\_last\_node}(\mathsf{node})$$

Key added. The transaction only affects the list by adding the given key:

$$\begin{aligned}\mathsf{key\_added}(\mathsf{key}, \mathsf{node\_cs}, \mathsf{node\_mint}) := \Big(&\mathsf{without\_lovelace}(\mathsf{node\_mint}) \\ &\equiv \mathsf{from\_asset}(\mathsf{node\_cs}, \mathsf{serialize\_key}(\mathsf{key}), 1)\Big)\end{aligned}$$

Key removed. The transaction only affects the list by removing the given key:

$$\begin{aligned}\mathsf{key\_removed}(\mathsf{key}, \mathsf{node\_cs}, \mathsf{node\_mint}) := \Big(&\mathsf{without\_lovelace}(\mathsf{node\_mint}) \\ &\equiv \mathsf{from\_asset}(\mathsf{node\_cs}, \mathsf{serialize\_key}(\mathsf{key}), -1)\Big)\end{aligned}$$

Key membership. The given key is a member of the list, proved by the existence of a witness node that satisfies the following predicate for the key.

$$\mathsf{is\_member}(\mathsf{key}, \mathsf{node}) \coloneqq (\mathsf{key} \equiv \mathsf{node.datum.key})$$

- When the above predicate is satisfied by a transaction input, it means that the key is a member of the list immediately before the transaction.
- When the above predicate is satisfied by a transaction output, it means that the key is a member of the list immediately after the transaction.

Key non-membership (ordered lists only!). The given key is *not* a member of the list, as proved by the existence of witness a node that satisfies the following predicate for the key:

$$\begin{aligned}
&\mathsf{is_non_member}(\mathsf{None}, _\mathsf{node}) \coloneqq \mathsf{False} \\
&\mathsf{is_non_member}(\mathsf{Some}(\mathsf{key}), \mathsf{node}) \coloneqq \\
&\qquad \Big(\mathsf{is_root_node}(\mathsf{node}) \vee (\mathsf{node.datum.key} < \mathsf{Some}(\mathsf{key}))\Big) \\
&\qquad\qquad \wedge \Big(\mathsf{is_last_node}(\mathsf{node}) \vee (\mathsf{Some}(\mathsf{key}) < \mathsf{node.datum.link})\Big)
\end{aligned}$$

- When the above predicate is satisfied by a transaction input, it means that the key is not a member of the list immediately before the transaction.
- When the above predicate is satisfied by a transaction output, it means that the key is not a member of the list immediately after the transaction.

In human terms, the key non-membership proof describes four possible scenarios in which a key is not a member of a key-ordered list:

1. The list is empty.
2. The root node links to a node with a higher key than the given key, which precludes the given key's membership because all subsequent nodes visited after this node will have monotonically increasing keys.
3. The last node has a key lower than the given key, which precludes the given key's membership because all nodes visited before the last node have even smaller keys.
4. A non-root non-last node has a lower key but links to a higher key than the given key, which precludes the given key's membership because all nodes before this node have smaller keys, and all nodes after this node have larger keys.

For a key-unordered list, the only way to prove the non-membership of a key is to iterate over the entire list in the transaction inputs, which quickly becomes infeasible for lists larger than the empty list. The `is_non_member` predicate does *not* apply to key-unordered lists.

A.1.6 Indexing into a list

The main way to index into a list is by key. Every linked list, whether key-ordered or key-unordered (if its keys are unique), has a canonical surjective mapping from its domain of all possible keys onto the nodes it contains:

- If a given key is a member of the list, it is mapped to the node that proves its membership.
- Otherwise, the key is mapped to the node that proves its non-membership.

In the offchain context, indexing by key requires traversing the list to the node that proves the key's membership or non-membership. However, indexing by key is cheap in the onchain context because only that single node needs to be an input to the transaction.

The other way to index into a list is by position, which requires traversing the list to the node at that position in both the offchain and onchain contexts. The root and last nodes are the cheapest to reach because they intrinsically define their positions within the list, so inspecting any other nodes is unnecessary.

The first and second-last nodes are the next cheapest to reach because they are adjacent to the root and last nodes, respectively. This means that the root node must be an input when indexing into the first node, while the last node must be an input when indexing into the second-last node. The farther a node is from the root or last node, the larger this chain of inputs grows to establish its position in the list.

A.1.7 Onchain traversal of a list

In the onchain context, read-only traversal through a list is more efficient backwards than forwards when the only thing that the traversal needs to know about the previously visited node is its key. In that case, backward traversal only needs the current node to be a transaction input because the current node's `link` field matches the key of the previously visited node.

By contrast, read-only forward traversal requires both nodes to be transaction inputs because it only knows which node it should be visiting by inspecting the previously visited node's `link` field. Of course, the traversal state could instead keep a copy of the `link` field of the previously visited node, but that copy can become stale if the list is not kept immutable during the traversal.

On the other hand, forward traversal is more efficient when the list is destructively folded into an accumulator that is stored at the root node. In that case, the root node needs to be updated anyway, and it always points to the next node that the traversal should visit.

A.1.8 Key-unordered linked list

The key-unordered linked list data structure does *not* keep its nodes ordered by their keys. This means that a new node cannot be added to the list based on its key because the list does not require the key to be positioned after any particular node in the list.

This leaves only adding new nodes to the list at specific absolute positions. We restrict these positions even further to just the beginning and end of the list because other positions would require increasingly larger chains of reference inputs to establish the new node's position.

Therefore, the key-unordered linked list data structure allows new nodes to be added to the beginning or end of the list, while non-root nodes can be removed anywhere in the list. It enforces the following properties:

1. The root node exists continuously until the list is initialized.
2. If the keys in the list are unique, then each node has only one node linking to it.
3. If the keys in the list are unique, then traversal through the list is deterministic.

State transition rules

Init. Initialize an empty list. Conditions:

1. The transaction's sole effect on the list is to add the root key.

$$\text{key\_added}(\text{None}, \text{node\_cs}, \text{node\_mint})$$

2. The list must be empty after the transaction, as proved by an output `root_node` that holds the minted root node NFT.

$$\text{is\_empty\_list}(\text{root\_node}) \land \text{has\_token}(\text{root\_node}, \text{node\_cs}, \text{``Node''})$$

3. The `root_node` must not contain any other non-ADA tokens.

The above conditions imply the following:

- `node_outputs` must be a singleton. This is implied by the list being empty after the transaction, the root node NFT being minted, and no other node tokens being minted or burned.
- `node_inputs` is empty. This is implied by the root node NFT of the list being minted in the transaction. If list keys are unique, no other node can exist before the root node. This is enforced by the other state transition rules, which (inductively) require the existence of the root node before and after any other node NFTs are created or burned.

The Init rule cannot enforce that the root node utxo is sent to the spending validator that contains the state integrity rules of the list because that spending validator is parametrized on the minting policy that includes the Init rule.

> ⚠ Offchain code for the list-initialization transaction MUST send the root node to the list's spending validator. Otherwise, the linked list can be corrupted.

Deinit. Deinitialize an empty list. Conditions:

1. The transaction's sole effect on the list is to remove the root key.

$$\text{key\_removed}(\text{None}, \text{node\_cs}, \text{node\_mint})$$

2. The list must be empty before the transaction, as proved by an input `root_node` that holds the minted root node NFT.

$$\text{is\_empty\_list}(\text{root\_node}) \land \text{has\_token}(\text{root\_node}, \text{node\_cs}, \text{``Node''})$$

The above conditions imply the following:

- `node_inputs` must be a singleton. This is implied by the empty list before the transaction.
- `node_outputs` must be empty. This is implied by the condition that the root node NFT is burned and that no other node tokens are minted or burned.

Prepend (unsafe). Prepend a new node to the beginning of the list. Grouped conditions:

- Verify the mint:
  1. Let `key_to_prepend` be the key being prepended.
  2. The transaction's sole effect on the list is to add `key_to_prepend`.

$$\mathsf{key\_added}(\mathsf{key\_to\_prepend}, \mathsf{node\_cs}, \mathsf{node\_mint})$$

- Verify the inputs:
  3. `node_inputs` must be a singleton. Let `anchor_node_input` be its sole node.
  4. `anchor_node_input` must be the root node of the list.
- Verify the outputs:
  5. `node_outputs` must have exactly two nodes:
     - `prepended_node`
     - `anchor_node_output`
  6. `key_to_prepend` must be a member of the list after the transaction, as witnessed by `prepended_node`.

$$\mathsf{is\_member}(\mathsf{key\_to\_prepend}, \mathsf{prepended\_node})$$

  7. `anchor_node_input` and `prepended_node` must match on the `link` field. In other words, they must both link to the same key.
  8. `anchor_node_output` must link to `key_to_prepend`.
  9. `prepended_node` must not contain any other non-ADA tokens.
- Verify immutable data:
  10. `anchor_node_input` must match `anchor_node_output` on address, value, and datum except for the `link` field.
  11. `prepended_node` must match `anchor_node_output` on address.

This rule is considered unsafe because it does *not* enforce key uniqueness or key order in the list. It merely assumes that the key being prepended is unique.

> ⚠ An application using a key-unordered list MUST only add nodes with unique keys to the list. Duplicate keys break the linked list data structure.

Append (unsafe). Append a new node to the end of the list. Conditions:

- Verify the mint:
  1. Let `key_to_append` be the key being appended.
  2. The transaction's sole effect on the list is to add `key_to_append`.

$$\mathsf{key\_added}(\mathsf{key\_to\_append}, \mathsf{node\_cs}, \mathsf{node\_mint})$$

- Verify the inputs:
  3. `node_inputs` must be a singleton. Let `anchor_node_input` be its sole node.
  4. `anchor_node_input` must be the last node of the list before the transaction.
- Verify the outputs:
  5. `node_outputs` must have exactly two nodes:
     - `appended_node`
     - `anchor_node_output`
  6. `key_to_append` must be a member of the list after the transaction, as witnessed by `appended_node`.

$$\mathsf{is\_member}(\mathsf{key\_to\_append}, \mathsf{appended\_node})$$

  7. `appended_node` must be the last node of the list after the transaction.
  8. `anchor_node_output` must link to `key_to_append`.
  9. `appended_node` must not contain any other non-ADA tokens.
- Verify immutable data:
  10. `anchor_node_input` must match `anchor_node_output` on address, value, and datum except for the `link` field.
  11. `appended_node` must match `anchor_node_output` on address.

This rule is considered unsafe because it does *not* enforce key uniqueness or key order in the list. It merely assumes that the key being appended is unique.

> ⚠ An application using a key-unordered list MUST only add nodes with unique keys to the list. Duplicate keys break the linked list data structure.

Remove. Remove a non-root node from the list. Conditions:

- Verify the mint:
  1. Let `key_to_remove` be the key being removed.

2. The transaction's sole effect on the list is to remove `key_to_remove`.

$$\text{key\_removed}(\text{key\_to\_remove}, \text{node\_cs}, \text{node\_mint})$$

- Verify inputs:

3. `node_inputs` must have exactly two nodes:
   - `removed_node`
   - `anchor_node_input`
4. `key_to_remove` must be a member of the list before the transaction, as witnessed by `removed_node`.

$$\text{is\_member}(\text{key\_to\_remove}, \text{removed\_node})$$

5. `anchor_node_input` must link to `key_to_remove`.

- Verify outputs:

6. `node_outputs` must be a singleton. Let `anchor_node_output` be its sole node.
7. `anchor_node_output` and `removed_node` must match on the `link` field. In other words, they must both link to the same key.

- Verify immutable data:

8. `anchor_node_input` must match `anchor_node_output` on address, value, and datum except for the `link` field.

A.1.9 Key-ordered linked list

The key-ordered linked list data structure replaces the unsafe Append rule of the key-unordered linked list with the safe Insert rule. It re-uses the rest of the key-unordered linked list rules, as these rules cannot cause a key-ordered list to become key-unordered. It enforces the following properties:

1. The root node exists continuously until the list is deinitialized.
2. Each node has a unique key.
3. Each key has only one node linking to it.
4. Traversal through the list is deterministic and follows key-ascending order.

State transition rules

Init. Same as Init for key-unordered lists.

> Offchain code for the list-initialization transaction MUST send the root node to the list's spending validator. Otherwise, the linked list can be corrupted.

Deinit. Same as Deinit for key-unordered lists.

Prepend (safe). Same as Prepend for key-unordered lists, but with an additional condition:

- `key_to_prepend` must *not* be a member of the list before the transaction, as witnessed by `anchor_node_input`.

$$\text{is\_not\_member}(\text{key\_to\_prepend}, \text{anchor\_node\_input})$$

The above condition enforces key uniqueness and order in the list, making this version of Prepend safe.

Append (safe). Same as Append for key-unordered lists, but with an additional condition:

- `key_to_append` must *not* be a member of the list before the transaction, as witnessed by `anchor_node_input`.

$$\text{is\_not\_member}(\text{key\_to\_append}, \text{anchor\_node\_input})$$

The above condition enforces key uniqueness and order in the list, making this version of Append safe.

Insert. Insert a node into the list. Grouped conditions:

- Verify mint:
  1. Let `key_to_insert` be the key being inserted.
  2. The transaction's sole effect on the list is to add `key_to_insert`.

$$\text{key\_added}(\text{key\_to\_insert}, \text{node\_cs}, \text{node\_mint})$$

- Verify inputs:
  3. `node_inputs` must be a singleton. Let `anchor_node_input` be its sole node.
  4. `key_to_insert` must *not* be a member of the list before the transaction, as witnessed by `anchor_node_input`.

$$\text{is\_not\_member}(\text{key\_to\_insert}, \text{anchor\_node\_input})$$

- Verify outputs:
  5. `node_outputs` must have exactly two nodes:

    - inserted_node
    - anchor_node_output

6. key_to_insert must be a member of the list after the transaction, as witnessed by inserted_node.

$$\text{is\_member}(\text{key\_to\_insert}, \text{inserted\_node})$$

7. anchor_node_output must link to key_to_insert.

8. anchor_node_input and inserted_node must match on the link field. In other words, they must both link to the same key.

9. inserted_node must not contain any other non-ADA tokens.

- Verify immutable data:

10. anchor_node_input must match anchor_node_output on address, value, and datum except for the link field.

11. inserted_node must match anchor_node_output on address.

Unlike the Append rule, the Insert rule is safe because it enforces key uniqueness and key order in the list:

- Key uniqueness is achieved by the inserted key's pre-transaction non-membership.
- Key order is achieved by requiring both anchor_node_input and inserted_node to link to the same key. The key non-membership condition implies that this common linked key is higher than key_to_insert, which means that inserted_node complies with the key order.

Remove. Same as Remove for key-unordered lists.

A.1.10 Instantiate a linked list in an application

An application that uses a linked list should ensure that it uses the appropriate state transition rules when creating/destroying node utxos of the list or otherwise modifying the key or link field value of any nodes. The application can attach additional rules that constrain the circumstances under which it allows list state transitions based on the list subcontext or the full transaction context. Moreover, the application has complete and exclusive control over the app-specific data field of list nodes, for which it also defines the data type.

As a general good practice, applications should limit the number of different non-node tokens and the size of app-specific data placed into node utxo when new nodes are inserted and when app-specific data is modified. This avoids token dust and large data attacks that could result in deadlock when certain list operations cannot fit within transactions' compute, memory, or space constraints.

Midgard uses linked lists throughout its onchain architecture:

- Registered operators (Section 3.2.2)
- Active operators (Section 3.2.3)

- Retired operators (Section 3.2.4)
- State queue (Section 3.4)
- Fraud proof catalogue (Section 4.1)

A.1.11 Different data in root node

An application that needs to store different data in the root node than in non-root nodes of a list can use an app-specific type with multiple constructors (i.e., a sum type) in the `data` field of the list's `NodeDatum`.

$$\begin{aligned} \mathsf{MyNodeDatum} &:= \mathsf{NodeDatum}(\mathsf{MyAppData}) \\ \mathsf{MyAppData} &:= \mathsf{Root}(\mathsf{MyRootData}) \mid \mathsf{Node}(\mathsf{MyNodeData}) \end{aligned}$$

The application's onchain code should ensure that root and non-root nodes always use their respective constructors.

A.1.12 Parallel insertions in key-ordered lists

For keys randomly sampled from an approximately uniform distribution (e.g., cryptographic public keys for wallets), insertion/removal from a key-ordered linked list is parallel to a degree proportional to the list's size, with parallelism increasing as the list grows.

An application may expect highly parallel traffic (e.g., from simultaneous interactions of independent users) before its list grows from its initially small size. This can be alleviated by using "separator" nodes in the list, which boost the list's parallelism by virtually occupying specific keys. Ideally, separators should be evenly spaced throughout the key space of the list.

$$\begin{aligned} \mathsf{MyNodeDatum} &:= \mathsf{NodeDatum}(\mathsf{MyAppDataWithSeps}) \\ \mathsf{MyAppDataWithSeps} &:= \mathsf{Root}(\mathsf{MyRootData}) \mid \mathsf{Node}(\mathsf{MyNodeData}) \mid \mathsf{Separator} \end{aligned}$$

Suppose the application needs to insert a node at a key occupied by a separator node. In that case, it will instead modify the node contents (see Appendix A.1.3) of the separator node to the intended `data` field value of the inserted node. Otherwise, node insertion works as usual.

The application can insert or remove separators as needed during the lifecycle of the list to regulate its parallelism.

A.2 Single-threaded state machine

A.2.1 Utxo representation

Each single-threaded state machine's current state is represented in the blockchain ledger as a single utxo:

- The spending validator of the utxo defines the possible transitions out of the current state.
- The utxo value contains a thread token corresponding to the state machine.
- The datum contains the output that the machine emitted upon entering the current state.

Each state transition of the state machine is executed via a separate blockchain transaction:

- The state machine's thread token must be unique within the transaction context.[3]
- The before-state is represented by the transaction input that contains the thread token.
- The after-state is represented by the transaction output that contains the thread token.
- The state transition's input is represented by the redeemer provided to the before-state's spending validator. If the spending validator defines several state transitions, the redeemer selects one for the transaction.

A.2.2 Minting policy

The thread token's minting policy defines the state machine's initial states, initialization procedure, final states, and termination procedure. It also implicitly defines the state machine's subgraph of reachable states, as each initial state's spending validator defines the transitions out of that state and, inductively, all further transitions out of the resulting states. Redeemers:

Initialize. Mint the thread token and send it to the spending validator address of one of the state machine's initial states. This redeemer receives an initial input that selects the initial state and provides additional information that can be referenced in subsequent state transitions.

The thread token's name should indicate the selected initial state and include a hash of the reference information provided in the initial input. If the state machine is deterministic, its thread token name identifies its unique path from initialization to the current state.

Finalize. Terminate the state machine normally if it is in one of the final states. Burn the thread token and (if needed) perform cleanup actions that are universally needed when terminating normally from any final state.

Cancel. Terminate the state machine exceptionally from any state. Burn the thread token and (if needed) perform cleanup actions that are universally needed when terminating exceptionally from any state.

[3]This avoids double-satisfaction issues during onchain validation of state transitions, as any transition's before and after states can be uniquely identified in any transaction. However, the state machine's thread token does *not* need to be globally unique across the blockchain ledger — it may be desirable to run several machines simultaneously, evolving their states in independent transaction chains.

A.2.3 Spending validators

If the state machine is non-deterministic, some of its spending validators define multiple possible transitions out of some states. The redeemers provided to these spending validators select the state transitions out of those states. Furthermore, the redeemers may provide additional arguments so that the spending validators have the context to decide whether to allow the selected state transitions.

Each spending validator is custom-written to express the specific logic of the possible transitions out of its state. For each of these transitions, the spending validator must include a condition that verifies the transformation of the input state into the corresponding output state:

$$\begin{aligned} \mathsf{verify\_transition}_{ij} &: (\mathsf{Input}_i, \mathsf{Output}_j, ..\mathsf{Args}_j) \to \mathsf{Bool} \\ \mathsf{verify\_transition}_{ij}(\mathsf{i}, \mathsf{o}, ..\mathsf{args}) &\coloneqq \left(\mathsf{transition}_{ij}(\mathsf{i}, ..\mathsf{args}) \equiv \mathsf{o}\right) \\ \mathsf{transition}_{ij} &: (\mathsf{Input}_i, ..\mathsf{Args}_j) \to \mathsf{Output}_j \end{aligned}$$

A.2.4 Compilation

Each spending validator can be either statically or dynamically parametrized on the spending validators into which its outbound state transitions lead.

- Static parametrization is preferred for state transitions that are expected to occur more frequently in typical executions of the state machine. A fancy way of expressing this is that state transitions should be statically parametrized along the maximally weighted acyclic subgraph of the state graph.
- Dynamic parametrization should be used for all other state transitions because statically parametrizing them would cause circular compilation dependencies on more preferred state transitions.

Dynamic parametrization means that the spending validator requires a reference input that indicates the addresses of the spending validators on which it dynamically depends. This reference input is crucial for the integrity of the state machine's state graph, so secure governance mechanisms should control its creation/modification.

A.2.5 Example

Consider a simplified model of the git pull-request (PR) workflow, with the following states:

Draft (initial state). The drafter is implementing a feature or bug fix in a repository branch. Transitions:

> **Update.** The drafter updates the branch by adding some git commits. Next state: Draft.
>
> **Request review.** The drafter requests a review for the branch. Next state: Testing.

Testing. The test suite is executing. Transitions:

> **Fail.** The branch fails its test suite. Next state: Draft.
>
> **Pass.** The branch passes its test suite. Next state: Review.

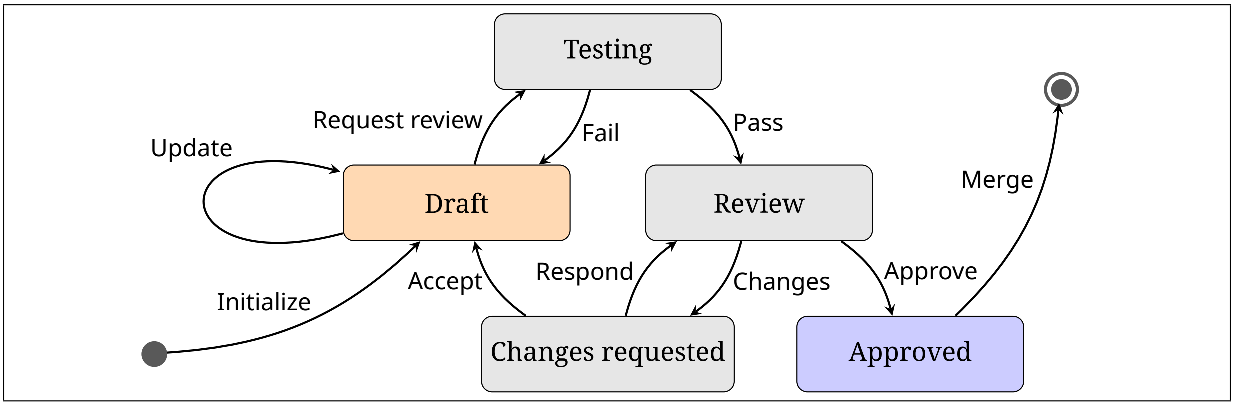

Figure A.1: State diagram for a simplified git PR workflow.

Review. The reviewer is deciding whether the branch should merge into the main branch. Transitions:

> **Approve.** The reviewer approves the branch to be merged into the repository's main branch. Next state: Merged.
>
> **Changes.** The reviewer requests some changes to the branch. Next state: Changes requested.

Changes requested. The drafter is considering the reviewer's feedback. Transitions:

> **Respond.** The drafter responds to the reviewer, arguing that changes are not required. Next state: Review.
>
> **Accept.** The drafter accepts the reviewer's change requests. Next state: Draft.

Approved (final state). The branch is merged into the main branch. Transitions:

> **Merge.** The state machine terminates normally from the final state. The branch is merged into the main branch and then deleted.

The onchain state machine representation of the above git PR model uses one minting policy and five spending validators. The minting policy:

- Defines Draft as the initial state.
- Assigns the state machine a token name corresponding to the commit hash of the base branch of the PR.[4]
- Defines Merged as the final state.
- Updates the state of the main branch when the PR branch is merged.

The spending validators define the transitions out of their corresponding states. The state datum types include the PR's current commit hash and other information relevant to their outbound transitions. For example, the spending validator for the Draft state has redeemers to validate two state transitions:

Update. Go to the Draft state. Update the PR commit hash and resolve any accepted change requests that the update addresses.

[4] In this simplified model, the base branch cannot be changed for a PR.

Request review. Go to the Testing state. Ensure that no accepted change requests remain.

We can also add a Cancel redeemer to every spending validator and the minting policy. In the git PR workflow, this state transition would reflect the fact that a PR can be closed at any time. Some additional logic may be needed in each of these redeemers to properly dispose of the PR after it is closed.

Appendix B

Midgard L1 transactions

This chapter specifies the L1 transactions that interact with Midgard's consensus protocol smart contracts. Whereas the onchain smart contract are specified in a modular way, where each validator is only concerned with its own validity conditions, the transactions specified in this chapter often interact with several of the onchain validators and must satisfy all of their conditions. In this way, the offchain code can be seen as the integration layer for the onchain code. TODO

Appendix C

Design and implementation considerations

This appendix chapter describes some considerations that we have taken into account while designing and implementing the Midgard protocol.

> It may be a good idea to move rationales out of the main chapters and into this appendix. Those chapters can focus on the "How?" of Midgard (useful for devs), while this appendix provide the "Why?" for architects and analysts.
> If this appendix chapter grows too big, we may eventually spin it out into separate "Design specification" and "Implementation guide" documents.

TODO

C.1 Protocol parameters

C.1.1 Midgard consensus protocol parameters

The following parameters must be set before Midgard is initialized. Their actual values will be determined once Midgard is fully implemented, tested, and benchmarks. However, for the reader's intuition, we provide approximate values that we expect for the parameters:

Parameter	Definition	Expected approx. value
`event_wait_duration`	Section 2.1	2–4 minutes
`fraud_prover_reward`	Section 3.2	30%–50% of the `required_bond` parameter
`maturity_duration`	Section 3.4	3–7 days
`registration_duration`	Section 3.2	1 day
`required_bond`	Section 3.2	50K–200K ADA
`settlement_resolution_duration`	Section 1.6	3–7 days
`shift_duration`	Section 3.1	1 hour
`slashing_penalty`	Section 3.2	50%–70% of the `required_bond` parameter

Table C.1: Midgard consensus protocol parameters.

Midgard does not use the Ouroboros consensus protocol, so it does not need to set the associated protocol parameters.

C.1.2 Midgard ledger parameters

Midgard's L2 transactions transition its ledger similarly to Cardano's ledger but with the exceptions defined in Section 1.5.2. Consequently, Midgard uses the same ledger-associated protocol parameters as Cardano but omits certain parameters as follows:

- No staking or governance parameters.
- No parameters for obsolete pre-Conway features.

C.1.3 Midgard fee structure

Midgard will collect fees from all L2 transactions, in a similar way to how fees are collected for Cardano L1 transactions. Furthermore, Midgard may collect fees for processing deposits and withdrawals, as these user events incur costs separate from L2 transactions. However, Midgard's fee parameters are expected to be orders of magnitude smaller than Cardano's fee parameters.

Midgard's L1 operating costs per block are fixed, regardless of the number and complexity of transactions in the block. Midgard's DA temporary storage costs per block are proportional to the number and size of transactions in the block, but these variable costs are orders of magnitude smaller than Cardano L1 storage costs. Midgard's revenue per block is proportional to the number and complexity of transactions in the block. Therefore, on a per-block basis, Midgard's fee revenue grows faster than its costs as the number of transactions in the block increases. This means that Midgard can sustain much lower L2 transaction fees once it attains a certain average level of L2 activity.

The specific values of Midgard's fee parameters will be determined before launch based on simulations, benchmarks, and community feedback.

C.1.4 Midgard network parameters

Technically, Midgard operator nodes do not need to communicate with each for consensus. Instead, they participate in the consensus protocol by interacting with Midgard's smart contracts on L1.

However, in practice, an offchain peer-to-peer gossip network between operators may help them run things a bit more smoothly for users. If so, there may be associated protocol parameters to help peers discover the network topology and communicate with other peers.

C.2 Protocol security

C.2.1 Malicious majority stake attack

A majority stake attack can allow the adversary to degrade the Liveness and Finality security properties of a blockchain protocol, if the adversary holds a sufficient percentage (e.g. 51%) of the network's primary resource (e.g. hashing power for Bitcoin, ADA for Cardano). In practical terms, this attack can allow the adversary to revert recently confirmed transactions and extract value from victims by preventing the reverted transactions from being restored to the ledger.

Midgard's consensus protocol is implemented entirely via Cardano L1 smart contracts. This means that an attack to revert any transaction that evolves the state of these L1 smart contract is as difficult as an attack against Cardano's finality security property, against which Cardano's Ouroboros consensus protocol is highly resistant. In simpler terms, Cardano's full ADA supply on L1 is the resource against which a majority stake attack must prevail to affect Midgard's L2 ledger.

Furthermore, every L2 transaction in Midgard is only confirmed after two consensus transactions occur on L1:

- First, the L2 transaction is included in a block header committed to Midgard's state queue.
- Second, the block header is merged to Midgard's confirmed state after waiting in the queue for the maturity period.

Both of the above L1 transactions must be reverted to revert a confirmed L2 transaction in Midgard. Due to the mandatory maturity period between the two L1 transactions, this amounts to a long-range attack to fork Cardano's blockchain before the maturity period began. However, according to the Ouroboros consensus protocol, Cardano nodes will always reject any forks that diverge from their local chain by more than 2160 blocks, which is approximately 12 hours (with very tight confidence bounds). As Midgard's maturity period protocol parameter will certainly be longer than 24 hours (see Appendix C.1), it will be impossible to revert both consensus transactions on L1 for a confirmed L2 transaction on Midgard.

Suppose the adversary succeeds in an attack against Cardano's finality property, reverting the second consensus transaction for a Midgard L2 transaction. This reversion will restore the block header to the beginning of the state queue, allowing it to be immediately re-merged to Midgard's confirmed state. Moreover, no conflicting block header can be merged to the confirmed state in its place. Thus, the adversary's attack has no practical effect other than slightly delaying the inevitable confirmation.

Suppose the adversary reverts the first consensus transaction for a Midgard L2 transaction. This reversion will remove the block header from the state queue. If the adversary is *not* colluding with the current operator in Midgard, then the current operator will simply recommit the block header to the state queue, nullifying any effects of the adversary's attack. If the adversary is colluding with the current operator, then the adversary's attack is redundant—the same effect is achieved if the current operator abuses its power to censor L2 transactions. However, Midgard has safeguards in place against such abuse (see Appendix C.2.2).

C.2.2 Operator abuse of power

Each operator has the exclusive privilege to commit blocks to the state queue and resolve nodes in the settlement queue during their assigned shifts (see Section 3.1), the duration of which is controlled by the `shift_duration` protocol parameter (see Appendix C.1). The current operator has discretion over whether and when to commit blocks to the state queue and the contents of those blocks. This discretion can be abused to censor L2 transactions, but Midgard's consensus protocol has safeguards in place against such abuse.

Midgard deposits and withdrawals are initiated via L1 smart contracts that assign definite inclusion times to them. An operator block is invalid if it contains these inclusion times in its event interval but fails to include the associated deposit or withdrawal events. This ensures that if operators continue committing blocks to Midgard's state queue, then they cannot ignore deposit and withdrawal events.

Midgard L2 transaction requests are typically submitted to operators via a publicly accessible API, and they can be ignored by operators. However, any user can escalate his L2 transaction request by posting a transaction order on L1. Similar to Midgard deposits and withdrawals, an L1 transaction order is assigned an inclusion time that guarantees its inclusion in a subsequent valid block.

If Midgard operators stop committing blocks at all to the state queue, then the inclusion times on their own cannot guarantee that deposits, withdrawals, and L2 transactions will be processed in a timely manner. However, for this extreme case, Midgard's consensus protocol includes the escape hatch mechanism, which allows a special non-optimistic block to be appended to the state queue by a non-operator. This block can include any deposits, withdrawals, and L2 transactions that are verified on L1 to comply with Midgard's ledger rules. This ensures that user funds cannot be stranded on Midgard even if its operators entirely stop committing blocks.

C.2.3 Replay attack

In a replay attack, the adversary repeats or delays a valid message to confuse the network and achieve a malicious outcome.

Midgard's entire consensus protocol is implemented as a set of smart contracts on Cardano L1 that evolve via L1 transactions. Each of these L1 transactions must spend at least one utxo, which means that Cardano's ledger rule against double-spending utxos prevent these transactions from being replayed.

However, Cardano's ledger rules do not directly see the contents corresponding to any hashes in the consensus protocol state on L1, such as the utxos, L2 transactions, deposits, and withdrawals in a block header. Nonetheless, Midgard has comprehensive ledger rules against the various ways in which utxos, L2 transactions, deposits, and withdrawals can be duplicated within a block or across blocks. Any operator that commits a block header to the state queue that contravenes these ledger rules forfeits his bond if the corresponding fraud proof is verified on L1 within the block's maturity period.

Another form of replay attack involves reusing the same confirmed deposit or withdrawal event in the settlement queue to absorb more deposited funds than necessary into Midgard's reserve or release more funds than necessary from it for a withdrawal. Midgard prevents this by requiring an L1 utxo to be created on L1 for every deposit and withdrawal, which must be spent whenever the corresponding deposit is absorbed or withdrawal is paid out.

List of Figures

1.1 A block's transition from a previous block's utxo set to a new utxo set. 4
1.2 A Merkle Patricia Trie example for a block's transactions. 5

3.1 Midgard time model. Each rectangle shows a block's time-validity interval. The color indicates the operator who must have committed the block. The top axis shows the operator shifts. The bottom axis shows the event intervals. 22

A.1 State diagram for a simplified git PR workflow. 81